

北京邮电大学

本科毕业设计（论文）



题目：基于强化学习的 Web 渗透测试大模型增强算法
的研究与仿真

姓 名 张艺翱
学 院 网络空间安全学院
专 业 信息安全
班 级 2022211801
学 号 2022211575
班内序号 13
指导教师 张 淼

2026 年 5 月

北 京 邮 电 大 学

本科毕业设计（论文）诚信声明

本人声明所呈交的毕业设计（论文），题目《基于强化学习的 Web 渗透测试大模型增强算法的研究与仿真》是本人在指导教师的指导下，独立进行研究工作所取得的成果。尽我所知，除了文中特别加以标注和致谢中所罗列的内容以外，论文中不包含其他人已经发表或撰写过的研究成果，也不包含为获得北京邮电大学或其他教育机构的学位或证书而使用过的材料。

申请学位论文与资料若有不实之处，本人承担一切相关责任。

本人签名：_____ 张艺翱 _____ 日期：_____ 2026 年 5 月 _____

关于论文使用授权的说明

本人完全了解并同意北京邮电大学有关保留、使用学位论文的规定，即：北京邮电大学拥有以下关于学位论文的无偿使用权，具体包括：学校有权保留并向国家有关部门或机构送交学位论文，有权允许学位论文被查阅和借阅；学校可以公布学位论文的全部或部分内容，有权允许采用影印、缩印或其它复制手段保存。汇编学位论文，将学位论文的全部或部分内容编入有关数据库进行检索。（保密的学位论文在解密后遵守此规定）

本人签名：_____ 张艺翱 _____ 日期：_____ 2026 年 5 月 _____
导师签名：_____ 日期：_____ 2026 年 5 月 _____

基于强化学习的 Web 渗透测试大模型增强算法的研究与仿真

摘 要

随着 Web 应用在政企业务、云服务平台和互联网产品中的广泛部署，Web 系统攻击面持续扩大，渗透测试在发现漏洞、验证风险和提升系统安全性方面具有重要意义。传统 Web 渗透测试高度依赖人工经验，成本较高且难以规模化复用；基于规则或脚本的自动化方法又难以适应复杂业务逻辑、多阶段攻击链和动态环境反馈。近年来，大语言模型（Large Language Model, LLM）在自然语言理解、代码生成和工具调用方面表现出较强能力，为自动化 Web 渗透测试提供了新的技术路径。然而，Web 渗透测试本质上是一类长时程、多阶段、部分可观测的交互式决策任务，通用 LLM 仍容易出现动作格式漂移、无效重复尝试、错误恢复不足和终局反馈稀疏等问题。因此，如何通过强化学习（Reinforcement Learning, RL）提升 LLM 在 Web 渗透测试场景中的持续行动能力和环境反馈适应能力，是本文研究的核心问题。

本文围绕“基于强化学习的 Web 渗透测试大模型增强算法”展开研究与仿真。首先，本文将 Web 渗透测试任务抽象为具有状态、动作、观测、奖励和终止条件的多轮交互决策过程，明确模型输入由任务描述、历史轨迹和当前观测共同构成，模型输出通过严格动作格式解析为单条可执行命令或提交动作。其次，本文构建从离线过程数据到在线交互环境的系统链路，将专家 Walkthrough 重构为 Thought-Command-Observation 三元组，并通过 Docker 封装受控 CTF 风格任务环境，实现命令执行、Observation 回流、轨迹记录和终局判定。再次，本文设计两阶段强化学习增强算法，Stage1 利用格式奖励和准确性奖励完成离线策略初始化，Stage2 将模型置于交互式环境中进行多轮轨迹采样 (rollout)，并采用群组相对策略优化 (Group Relative Policy Optimization, GRPO) 更新模型策略。针对终局奖励稀疏问题，本文进一步设计基于大模型评审器 (LLM Judge, 下文简称 Judge) 的进展奖励塑形机制，通过阶段评价、潜势差分、重复进展抑制和奖励上限等方式，为模型提供更可信的中间过程反馈。

实验部分以 AutoPenBench 和 Cybench 作为主要外部评测基准，从端到端成功率、子任务完成率、阶段消融、Judge 进展奖励和 Token 消耗等角度分析模型能力。实验结果表明，本文模型在 33 个 AutoPenBench 任务上取得 30.3% 的 Success Rate 和 38.5% 的 Subtask-Success；在 Cybench 上取得 17.5% 的 Unguided、17.5% 的 Subtask-Guided 和 46.8% 的 Subtasks-Solved。与不使用 Judge 进展评估的两阶段消融配置相比，AutoPenBench 的 Success Rate 从 24.2% 提升至 30.3%，Subtask-Success 从 33.4% 提升至 38.5%；Cybench 三项指标也从 15.0%、15.0% 和 42.8% 提升至 17.5%、17.5% 和 46.8%。结果说明，仅依赖通用推理能力或静态监督学习难以稳定完成多步 Web 渗透测试任务，而将专家过程数据、交互式环境反馈、强化学习策略优化和基于证据的中间进展评价结合起来，能够

有效提升模型的端到端任务完成能力。本文研究为面向 Web 渗透测试的大模型智能体训练、仿真评测和受控安全实验提供了一种可实现的技术路线。

关键词 Web 渗透测试 大语言模型 强化学习 进展奖励 LLM Judge

Research and Simulation of Large-Scale Web Penetration Testing Model Augmentation Algorithm Based on Reinforcement Learning

ABSTRACT

With the wide deployment of Web applications in enterprise systems, cloud services, and Internet platforms, the attack surface of Web systems continues to expand. Penetration testing plays an important role in vulnerability discovery, risk validation, and security assurance. Traditional Web penetration testing heavily relies on human expertise, which provides flexibility but also leads to high cost, long testing cycles, and limited scalability. Rule-based or script-based automation methods are difficult to generalize to complex business logic, multi-stage attack chains, and dynamic environment feedback. Recent large language models (LLMs) have shown strong capabilities in natural language understanding, code generation, and tool use, which creates new opportunities for automated Web penetration testing. However, Web penetration testing is essentially a long-horizon, multi-stage, partially observable interactive decision-making task. General-purpose LLMs still suffer from unstable action formats, repeated invalid attempts, weak error recovery, and sparse final rewards. Therefore, this thesis studies how to enhance the interactive Web penetration testing capability of LLMs through reinforcement learning.

This thesis presents a reinforcement-learning-based LLM enhancement method for Web penetration testing. First, the task is formulated as a multi-turn interaction process with states, actions, observations, rewards, and termination conditions. The model state consists of the task description, historical trajectory, and current observation, while the model output is parsed into a single executable command or submission action through a strict action interface. Second, a system pipeline is constructed from offline process data to online interactive environments. Expert Walkthroughs are reconstructed into Thought-Command-Observation triples, and controlled CTF-style environments are encapsulated with Docker to support command execution, observation feedback, trajectory logging, and final result verification. Third, a two-stage reinforcement learning algorithm is designed. Stage1 performs offline policy initialization with format and accuracy rewards, while Stage2 conducts multi-turn rollouts in executable environments and updates the policy with Group Relative Policy Optimization (GRPO). To alleviate sparse final rewards, a progress reward shaping mechanism based on an LLM Judge is further introduced. It provides intermediate feedback by stage evaluation, potential-based difference rewards, duplicate progress suppression, and reward caps.

Experiments are conducted on AutoPenBench and Cybench. The evaluation analyzes end-to-end success rate, subtask completion rate, stage ablation, Judge-based progress rewards, and token consumption. The proposed model achieves a 30.3% Success Rate and a 38.5% Subtask-

Success on 33 AutoPenBench tasks, and obtains 17.5% Unguided, 17.5% Subtask-Guided, and 46.8% Subtasks-Solved on Cybench. Compared with the two-stage ablation configuration without Judge-based progress evaluation conducted in this thesis, the Success Rate on AutoPenBench increases from 24.2% to 30.3%, and the Subtask-Success increases from 33.4% to 38.5%; the three Cybench metrics also improve from 15.0%, 15.0%, and 42.8% to 17.5%, 17.5%, and 46.8%. The results indicate that general reasoning ability or static supervised learning alone is insufficient for stable multi-step Web penetration testing. Combining expert process data, interactive environment feedback, reinforcement learning policy optimization, and evidence-based intermediate progress evaluation can effectively improve the end-to-end task completion capability of LLM-based agents. This thesis provides an implementable technical route for training, simulation, and controlled evaluation of LLM agents for Web penetration testing.

KEY WORDS Web penetration testing Large language model Reinforcement learning GRPO LLM Judge

目 录

第一章 绪论	1
1.1 研究背景	1
1.2 研究意义	1
1.3 国内外研究现状	2
1.4 研究内容与技术路线	4
1.5 主要工作与创新点	4
1.6 论文结构安排	5
第二章 相关理论与关键技术基础	6
2.1 Web 渗透测试与过程知识	6
2.1.1 多阶段任务特征	6
2.1.2 Walkthrough 与 TCO 表示	6
2.2 大语言模型智能体交互机制	7
2.2.1 提示词与动作接口	7
2.2.2 推理文本的作用与局限	7
2.3 强化学习基本原理	7
2.3.1 奖励塑形与奖励作弊	8
2.4 参数高效微调与策略优化方法	8
2.5 评测基准与指标	9
2.6 本章小结	9
第三章 面向 Web 渗透测试的大模型增强任务建模与总体方案	10
3.1 问题定义与设计目标	10
3.2 总体方法组成	10
3.3 Web 渗透测试任务的 MDP 建模	11
3.3.1 状态空间	11
3.3.2 动作空间与终止条件	12
3.3.3 奖励信号	12
3.4 系统总体架构	13
3.5 两阶段增强方案	13
3.5.1 Stage1 离线知识内化	13
3.5.2 Stage2 在线交互强化	13
3.5.3 Judge 进展塑形	14
3.6 本章小结	14

第四章 测试环境与系统构建	15
4.1 系统需求分析	15
4.2 离线过程数据构建	15
4.2.1 TCO 三元组重构	15
4.2.2 Stage1 样本组织	16
4.3 在线任务数据构建	16
4.4 Prompt 与动作格式	17
4.5 多轮交互环境实现	17
4.6 训练与评测接口	18
4.7 本章小结	18
第五章 算法设计	19
5.1 算法目标与总体思路	19
5.2 GRPO 策略优化	19
5.3 Stage1 离线强化学习	20
5.3.1 格式奖励	20
5.3.2 准确性奖励	20
5.3.3 训练流程	21
5.4 Stage2 在线多轮轨迹优化	21
5.4.1 基础交互奖励	21
5.4.2 终局验证奖励	22
5.4.3 多轮损失掩码	22
5.5 Judge 进展奖励塑形	22
5.5.1 阶段集合与评价输入	23
5.5.2 Judge 输出结构	23
5.5.3 潜势差分奖励	24
5.5.4 重复进展抑制与负向标记惩罚	24
5.5.5 奖励上限与最终进展奖励	25
5.6 总奖励与完整流程	25
5.7 本章小结	27
第六章 实验设计与结果分析	28
6.1 实验目标与研究问题	28
6.2 实验环境与数据集	28
6.2.1 训练与评测环境	28
6.2.2 训练数据集	28
6.2.3 评测基准与任务划分	30

6.3	评价指标与实验配置	30
6.3.1	AutoPenBench 指标	30
6.3.2	Cybench 指标	31
6.3.3	评测预算与生成参数	31
6.4	对比模型与消融设置	32
6.4.1	外部对比模型	32
6.4.2	阶段与 Judge 消融设置	32
6.5	主实验结果	32
6.5.1	整体性能对比	32
6.5.2	指标差异分析	33
6.6	消融实验分析	34
6.6.1	两阶段训练与 Judge 进展评估贡献	34
6.6.2	子任务完成率与最终成功率关系	35
6.7	Token 消耗与推理效率分析	35
6.8	过程行为与失败原因分析	37
6.8.1	典型成功轨迹分析	37
6.8.2	失败原因分类	38
6.9	实验结论与讨论	39
6.9.1	主要结论	39
6.9.2	有效性与局限性	39
6.10	本章小结	40
第七章	总结与展望	41
7.1	全文工作总结	41
7.2	主要结论	41
7.2.1	两阶段训练能够提升端到端能力	41
7.2.2	局部进展不等于最终成功	41
7.2.3	奖励设计是长链路任务的关键	42
7.3	不足与局限	42
7.4	未来工作展望	42
7.5	本章小结	43
	参考文献	44
	致谢	46
	附录	47
	附录 1 缩略语与符号说明	47
	附录 2 关键 Prompt 与动作格式	48

附录 3 数据字段与训练配置	49
附录 4 奖励参数与评测指标	51

第一章 绪论

1.1 研究背景

随着政企信息系统、云服务平台和互联网业务的持续发展，Web 应用已经成为现代信息系统最重要的对外服务入口。用户认证、数据查询、业务审批、在线交易和内容发布等关键功能大量依赖 Web 接口提供服务。与此同时，系统结构从传统单体应用演化为前后端分离、微服务编排和多组件协同的复杂形态，攻击面随之扩大。SQL 注入、命令执行、文件上传、身份认证绕过、任意文件读取和反序列化等漏洞仍然是 Web 安全测试中的重点对象^[1-3]。

传统 Web 渗透测试高度依赖人工经验。测试人员通常需要在授权范围内完成资产识别、入口发现、参数分析、漏洞验证、利用扩展和结果复核等工作。该过程不是单步问答，而是由“观察环境、分析线索、执行动作、根据反馈修正方向”的循环构成。规则脚本和漏洞扫描器能够覆盖部分标准化漏洞，但面对复杂业务逻辑、隐藏入口和多阶段攻击链时，往往难以完成端到端推理与行动^[4]。

近年来，大语言模型（Large Language Model, LLM）在自然语言理解、代码生成、指令跟随和工具调用方面展现出较强能力，为自动化 Web 渗透测试提供了新的技术路径^[5-7]。LLM 可以理解题面、命令输出、错误信息和日志内容，并将高层目标拆解为目录枚举、文件读取、脚本编写、服务交互和结果提交等具体动作。然而，通用 LLM 并不能天然胜任渗透测试任务。Web 渗透测试本质上是长时程、多阶段、部分可观测的交互式决策问题，模型容易出现动作格式漂移、命令不可执行、重复试错、错误恢复不足和推理文本冗长等问题^[8-10]。

强化学习（Reinforcement Learning, RL）为解决上述问题提供了合适视角^[11]。与监督学习只学习参考答案不同，强化学习强调智能体在环境中通过试错获取奖励，并以最大化长期回报为目标优化策略。对于 Web 渗透测试任务而言，模型不仅要学习专家命令形式，还要学习如何根据真实 Observation 修正动作、恢复错误并在有限交互预算内完成目标。因此，本文研究的问题可以概括为：如何通过强化学习增强 LLM 在 Web 渗透测试场景中的持续行动能力、环境反馈适应能力和端到端任务完成能力。

1.2 研究意义

从工程意义看，强化学习增强的 Web 渗透测试智能体有助于降低安全测试对人工规则和高强度人工参与的依赖，适用于安全教学、靶场训练、批量评测和授权测试辅助等受控场景。与单纯脚本工具相比，LLM 能够处理自然语言任务描述和非结构化反馈；与单纯 Prompt 工程相比，强化学习能够让模型在真实反馈下形成更稳定的动作策略。

从研究意义看，该问题连接了网络安全、LLM Agent、强化学习、奖励塑形和交互式评测等多个方向。Web 渗透测试任务奖励稀疏、路径长、失败反馈复杂，若只依赖终局 Flag 奖励，模型很难从失败轨迹中学习；若中间奖励设计不当，又可能诱导模型通过

重复扫描或表面活跃获得虚假分数。因此，如何构造既保持最终目标导向、又能刻画真实阶段推进的奖励信号，是本文关注的核心问题。

1.3 国内外研究现状

自动化 Web 渗透测试相关研究大致可以分为三类，如表 1-1 所示。第一类是规则驱动方法，依赖漏洞特征、扫描器和利用脚本，优点是稳定、可解释、适合已知模式，缺点是泛化能力有限。第二类是大语言模型智能体（LLM Agent）方法，利用模型进行规划、工具调用和结果分析，能够处理更复杂的语义信息，但仍高度依赖提示词（Prompt）和外部规则维持行为稳定^[8-10,12]。第三类是强化学习增强方法，将模型放入可执行环境中，通过奖励信号优化多轮动作策略，能够更直接地面向端到端任务完成目标^[13-15]。

规则驱动方法是较早被工程化采用的自动化路线。此类方法通常围绕漏洞规则库、指纹识别、扫描插件和利用模块构建，能够在已知漏洞和标准化配置错误上取得较稳定效果。例如，常见漏洞扫描器和利用框架可以自动完成端口探测、目录枚举、指纹识别和部分漏洞验证。但这一路线的核心限制在于规则覆盖范围有限，面对需要业务语义理解、多步凭证传递或动态反馈调整的场景时，系统往往难以自动生成新的攻击链。

随着 LLM 在代码生成和工具调用方面的能力增强，研究者开始尝试将模型作为渗透测试智能体的规划核心。相关系统通常采用“观察、思考、行动”的循环，让模型读取题面、命令输出或网页响应，再生成下一步命令或工具调用。这类方法降低了人工编写规则的成本，也提升了对自然语言任务描述和非结构化日志的理解能力。然而，单纯依赖提示词约束仍然存在动作格式漂移、错误恢复不足、重复尝试和上下文遗忘等问题，模型是否真正完成任务仍需通过可执行环境检验。

强化学习为上述问题提供了进一步优化路径。相比监督学习只拟合专家样本，强化学习可以把模型置于环境反馈中，根据轨迹回报更新策略，使模型学习在失败后调整路径、在有限预算内推进任务。近年来，GRPO 等面向大模型的策略优化方法降低了训练 Critic 网络的复杂度，也使长文本、多轮轨迹优化更加可行。与此同时，过程奖励和奖励塑形研究表明，长链路任务不能只依赖终局成功信号，还需要设计受控的中间反馈，帮助模型区分真实推进、重复尝试和虚假进展。

已有研究表明，单纯增加链式思考（Chain-of-Thought, CoT）并不必然提升自动化渗透测试成功率^[15,16]。在多轮环境中，评价对象不是推理文本是否充分，而是模型能否将推理转化为可执行动作，并在失败后调整策略。当前研究仍面临四个主要不足：一是高质量多步专家过程数据不足；二是终局奖励稀疏导致学习信号弱；三是中间奖励容易被模型投机利用；四是训练接口和评测接口不一致会削弱结果可信度^[17-19]。

图 1-1 概括了本文的研究动机。通用大语言模型在渗透测试任务中既存在动作解析、错误恢复和复杂推理方面的问题，也缺少来自真实靶场过程的动手实践能力；单轮强化学习又难以覆盖多轮 Web 渗透测试链路。针对这些问题，本文采用“离线过程强化学习 + 在线交互强化 + 大模型评审器（LLM Judge，下文简称 Judge）进展评估”的路线，

表 1-1 现有研究路线与本文方法对比

研究路线	主要特点	优势	局限
规则与脚本驱动	依赖扫描器、规则库和利用模块	稳定、可解释、工程成熟	难以处理复杂上下文和未知路径
LLM Agent	通过提示词组织观察、思考和行动	语义理解强，工具调用灵活	动作格式和错误恢复不稳定
强化学习增强	在交互环境中根据奖励优化策略	能学习长期回报和反馈适应	环境构建与奖励设计成本高
本文方法	离线专家过程与在线交互强化结合	兼顾过程先验、真实反馈和端到端评测	仍受任务规模和计算资源限制

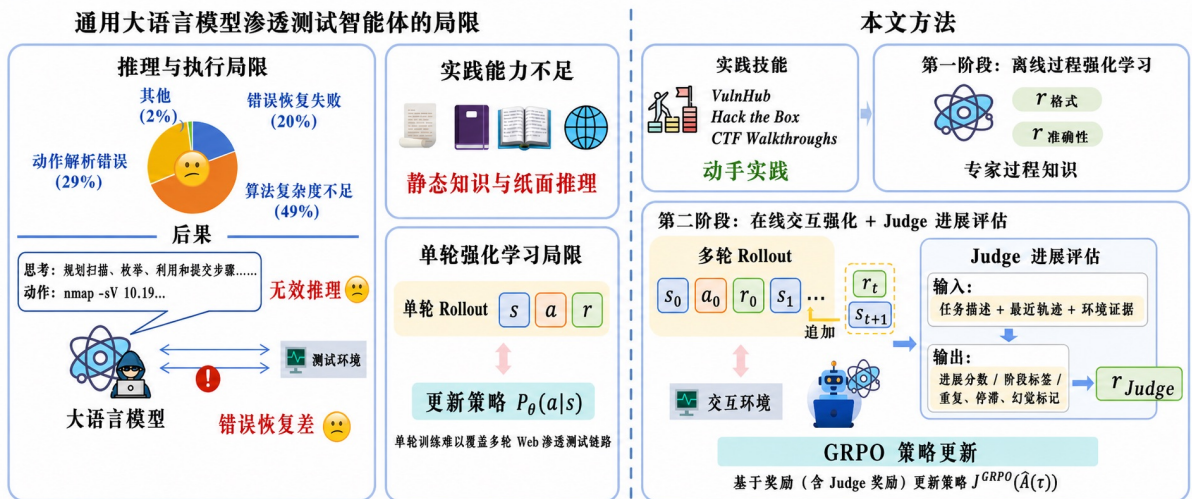


图 1-1 通用大语言模型渗透测试智能体的局限与本文方法思路

使模型先学习专家过程知识，再在可执行环境中根据真实反馈和中间进展信号优化策略。

1.4 研究内容与技术路线

围绕“基于强化学习的 Web 渗透测试大模型增强算法的研究与仿真”这一主题，本文主要开展以下工作。

- 建立 Web 渗透测试任务的多轮交互建模方法，将任务描述、历史轨迹、当前观测和元信息组织为状态，将模型输出解析为单个可执行动作或提交动作。
- 构建离线过程数据，将真实专家 Walkthrough 重构为 Thought-Command-Observation 三元组，使模型学习常见测试步骤、命令形式和反馈利用方式。
- 构建受控在线交互环境，通过 Docker 封装 CTF 风格任务，实现命令执行、Observation 回流、轨迹记录、奖励计算和终局判定。
- 实现两阶段强化学习训练流程。Stage1 通过离线样本完成策略初始化，Stage2 通过多轮轨迹采样（rollout）和群组相对策略优化（Group Relative Policy Optimization, GRPO）学习环境反馈下的策略调整能力。
- 重点设计基于 Judge 的进展奖励塑形机制，对阶段推进、重复尝试、停滞状态和幻觉性进展进行约束，以缓解终局奖励稀疏问题。
- 在 AutoPenBench 和 Cybench 上完成评测，在 AutoPenBench 上取得 30.3% 的 Success Rate 和 38.5% 的 Subtask-Success，在 Cybench 上取得 17.5%、17.5% 和 46.8% 的三项结果，并通过阶段消融分析两阶段训练与 Judge 进展评估的贡献。

本文技术路线可概括为“离线过程学习 + 在线交互强化 + 统一基准评测”。首先，利用专家 Walkthrough 提供基础行为先验；其次，将模型放入可执行环境中获取真实反馈；最后，用统一动作接口和评测预算衡量端到端能力。该路线的核心不是让模型一次性生成完整解题报告，而是让模型在每一轮基于证据选择下一步动作。

1.5 主要工作与创新点

本文主要工作和创新点体现在以下三个方面。其中，两阶段训练、GRPO 优化和交互式基准评测属于本文采用并工程实现的基础技术路线；本文重点展开的设计与验证集中在面向 Web 渗透测试轨迹的 Judge 进展奖励塑形及其与在线交互训练的结合。

- 构建面向 Web 渗透测试任务的两阶段大模型增强框架。该框架将专家过程学习与在线交互强化结合，缓解了单纯监督学习难以提升端到端任务完成能力的问题。

- 完成多轮 Web 渗透测试任务的强化学习建模与闭环实现。本文明确了状态、动作、观测、奖励和终止条件，并构建了从模型输出到环境执行再到轨迹记录的完整链路。
- 设计并接入基于 Judge 的进展奖励塑形方法。该方法以真实观测（Observation）和环境证据为依据，通过潜势差分、奖励上限、重复惩罚和停滞惩罚抑制奖励作弊，使中间反馈更接近真实任务推进，并通过有无 Judge 的消融实验验证其贡献。

1.6 论文结构安排

本文共分七章。第一章介绍研究背景、研究意义、研究现状和主要工作。第二章介绍 Web 渗透测试、LLM Agent、强化学习、GRPO 和评测指标等理论基础。第三章给出任务建模与总体方案。第四章介绍测试环境与系统构建。第五章详细说明两阶段强化学习算法和 Judge 进展奖励。第六章给出实验设计、主结果、消融分析和局限讨论。第七章总结全文并展望后续研究方向。

第二章 相关理论与关键技术基础

2.1 Web 渗透测试与过程知识

Web 渗透测试是指在合法授权前提下，从攻击者视角对目标 Web 应用及相关服务进行系统化安全验证的过程，其目标是发现可利用缺陷并评估其对机密性、完整性和可用性的影响^[1-3]。与单一漏洞检测不同，真实 Web 渗透测试通常包含信息收集、目标定位、漏洞验证、利用推进和结果复核等阶段，并且这些阶段会因环境反馈不断回退和调整。

2.1.1 多阶段任务特征

信息收集阶段的目标是建立对目标系统的基本认知，包括服务端口、目录结构、接口参数、身份认证方式和技术栈指纹等。漏洞验证阶段需要将“可能存在弱点”转化为“漏洞真实存在并可被利用”的证据。利用推进阶段则常常需要把多个局部线索串联为攻击链，例如先发现隐藏文件，再提取凭证，最后访问受限资源或提交 Flag。

对自动化系统而言，难点在于环境反馈通常不直接给出下一步动作。模型需要从命令输出、错误信息、文件变化和网页响应中抽取线索，并决定后续动作。若模型无法区分“无回显但可能成功”和“命令根本未执行”等细节，就难以形成稳定的利用策略。因此，Web 渗透测试天然适合被建模为多轮交互决策任务。

2.1.2 Walkthrough 与 TCO 表示

专家 Walkthrough 保留了测试人员在多步任务中的思维过程、命令选择和执行反馈，能够为模型提供比“题目到答案”更丰富的过程知识^[15]。本文将其实重构为 Thought-Command-Observation (TCO) 三元组，其中 Thought 表示当前步骤意图，Command 表示可执行动作，Observation 表示环境反馈。

表 2-1 Walkthrough 过程知识表示方式

字段	含义	作用
Thought	对当前线索和下一步目标的分析	提供过程推理示范
Command	专家实际执行的命令或提交动作	提供动作格式和命令先验
Observation	命令执行后的环境反馈	建立动作与反馈之间的关联

TCO 表示的价值在于，它把推理、动作和反馈绑定到同一决策步骤中。Stage1 离线训练可利用该结构学习专家步骤模式，Stage2 在线训练也可用同样结构记录模型轨迹、计算奖励并分析失败原因。

2.2 大语言模型智能体交互机制

LLM 具备自然语言理解、代码生成和工具调用能力，为自动化 Web 渗透测试提供了基础^[5,8,9]。在大语言模型智能体（LLM Agent）框架中，模型不再只是回答问题，而是在每一轮读取任务目标、历史轨迹和当前观测（Observation），生成下一步动作。该过程可表示为

$$a_t \sim \pi_{\theta}(\cdot | q, h_{t-1}, o_t) \quad \text{式 (2-1)}$$

其中， q 表示任务描述， h_{t-1} 表示历史轨迹， o_t 表示当前观测， a_t 表示模型动作。

2.2.1 提示词与动作接口

提示词（Prompt）是连接模型与外部环境的关键接口^[6,7]。对于 Web 渗透测试任务，Prompt 不仅要描述目标，还必须规定输出格式。若模型一轮输出多个命令、解释性文本或不可执行建议，环境就无法稳定解析动作（Action）。本文采用“单轮单动作”的受控接口，使模型完整回复包含 Reflection、Research Plan and Status、Thought 和 Action 字段，而环境只执行 Action 字段。

表 2-2 基于提示词的 Web 渗透测试智能体输入输出结构

组成	说明
任务描述	给出测试目标、初始资源和提交要求
历史轨迹	保存已执行动作和关键 Observation
当前观测	反馈最新命令输出、错误信息或状态变化
Action 字段	给出唯一可解析、可执行的命令或提交动作

2.2.2 推理文本的作用与局限

显式 CoT 能帮助模型组织分析，但推理文本越长并不必然意味着任务成功率越高^[15,16]。在 Web 渗透测试中，评价对象是模型能否在有限轮次内生成有效动作并推进环境状态。如果模型反复解释错误假设却不执行关键动作，长推理反而会消耗 Token、污染上下文并延续错误路径。因此，本文更关注推理和行动的一致性。

2.3 强化学习基本原理

强化学习研究智能体如何在环境中通过试错最大化长期累计回报^[11]。一个标准马尔可夫决策过程（Markov Decision Process, MDP）可表示为

$$\mathcal{M} = (\mathcal{S}, \mathcal{A}, P, R, \gamma) \quad \text{式 (2-2)}$$

其中, $\mathcal{S}$ 为状态空间, $\mathcal{A}$ 为动作空间, P 为状态转移函数, R 为奖励函数, γ 为折扣因子。

在 Web 渗透测试中, 模型只能观察到环境的局部反馈, 严格来说更接近部分可观测任务。工程实现中可以将任务描述、历史轨迹和当前 Observation 合并为扩展状态, 使其近似满足策略学习需要。一次交互轨迹可写为

$$\tau = (s_0, a_0, r_0, s_1, a_1, r_1, \dots, s_T) \quad \text{式 (2-3)}$$

策略优化目标是最大化期望回报:

$$J(\theta) = \mathbb{E}_{\tau \sim \pi_\theta} \left[\sum_{t=0}^T \gamma^t r_t \right] \quad \text{式 (2-4)}$$

2.3.1 奖励塑形与奖励作弊

长链路 Web 渗透测试的终局奖励高度稀疏, 模型可能在大量失败轨迹中得不到有效学习信号。奖励塑形 (Reward Shaping) 通过引入中间反馈缓解这一问题^[20]。典型势函数塑形可表示为

$$F(s_t, s_{t+1}) = \gamma \Phi(s_{t+1}) - \Phi(s_t) \quad \text{式 (2-5)}$$

其中, $\Phi(\cdot)$ 为状态势函数。

奖励塑形必须避免奖励作弊。若系统只奖励命令成功退出, 模型可能反复执行无意义命令; 若只奖励发现新文本, 模型可能制造噪声输出。因此, 本文强调中间奖励必须绑定真实环境证据, 并通过差分、截断和重复惩罚限制其影响。

2.4 参数高效微调与策略优化方法

直接微调 LLM 全部参数成本较高。低秩适配 (Low-Rank Adaptation, LoRA) 通过冻结原始权重 W_0 , 仅训练低秩增量矩阵来降低显存和计算开销^[21]:

$$W = W_0 + \Delta W, \quad \Delta W = BA, \quad \text{rank}(BA) \ll \min(d, k) \quad \text{式 (2-6)}$$

近端策略优化 (Proximal Policy Optimization, PPO) 通过裁剪策略比率限制更新幅度, 提升训练稳定性^[22]。设

$$\rho_t(\theta) = \frac{\pi_\theta(a_t | s_t)}{\pi_{\theta_{\text{old}}}(a_t | s_t)} \quad \text{式 (2-7)}$$

则其核心目标包含 $\rho_t(\theta) \hat{A}_t$ 与裁剪项之间的最小值。

群组相对策略优化 (Group Relative Policy Optimization, GRPO) 进一步使用同一输入下多条候选样本的组内相对回报估计优势, 避免单独训练 Critic 网络^[13,14]。设同一任

务采样 K 条轨迹，回报为 $R_1, \dots, R_K$ ，则优势可写为

$$\hat{A}_k = \frac{R_k - \mu_R}{\sigma_R + \epsilon} \quad \text{式 (2-8)}$$

其中， μ_R 和 σ_R 分别为组内回报均值与标准差。对于多轮 Web 渗透测试任务，GRPO 可以在轨迹级回报上比较不同尝试的优劣，并通过损失掩码只更新 assistant 侧生成 Token，避免模型学习复述环境 Observation。

2.5 评测基准与指标

自动化渗透测试模型需要在可执行环境中验证能力。AutoPenBench 是面向自动化渗透测试智能体的交互式评测基准，要求模型在受控环境中多轮执行动作并提交结果^[19]。Cybench 和 InterCode-CTF 等环境也体现了从静态问答转向真实交互评测的趋势^[17,18]。

本文重点使用 Success Rate (SR) 和 Subtask-Success (SS) 两个指标。设任务总数为 N ，成功任务数为 N_{succ} ，则

$$\text{SR} = \frac{N_{\text{succ}}}{N} \quad \text{式 (2-9)}$$

设关键子任务总数为 M ，模型完成的子任务数为 M_{sub} ，则

$$\text{SS} = \frac{M_{\text{sub}}}{M} \quad \text{式 (2-10)}$$

表 2-3 自动化渗透测试常用评价指标说明

指标	含义
Success Rate	模型在限定预算内完成完整任务的比例
Subtask-Success	模型完成关键中间步骤的比例
平均交互步数	衡量模型完成任务或失败前的交互成本
合法动作比例	衡量模型输出能否被环境稳定解析和执行
Token 消耗	衡量推理文本和动作生成的计算成本

2.6 本章小结

本章概括了后续研究所需的理论与技术基础。Web 渗透测试具有多阶段、强反馈和长链路特征；LLM Agent 需要通过严格 Prompt 与动作接口连接外部环境；强化学习为环境反馈下的策略优化提供了理论框架；LoRA、PPO 和 GRPO 为低成本、稳定的大模型策略训练提供了工程基础；AutoPenBench 等交互式基准则为评测端到端能力提供了统一口径。

第三章 面向 Web 渗透测试的大模型增强任务建模与总体方案

3.1 问题定义与设计目标

本文关注的问题不是让模型回答单个安全知识问题，而是让模型在受控 Web 渗透测试环境中围绕目标持续采取有效动作，并在失败、噪声和稀疏反馈条件下逐步推进任务^[8-10]。与普通问答相比，该任务具有长时程、部分可观测、过程依赖和奖励稀疏等特点，因此需要从强化学习角度进行建模。

本文总体设计目标包括四点。第一，构建统一动作接口，使模型输出能够稳定解析为单个命令或提交动作。第二，构建可执行环境，使模型动作能够得到真实 Observation。第三，构建分阶段训练流程，使模型先获得专家过程先验，再通过在线交互学习错误恢复。第四，构建可靠奖励结构，使训练信号既保持最终成功导向，又能反映真实中间进展。

3.2 总体方法组成

本文总体方案由过程数据构造、交互环境封装、两阶段强化学习、奖励塑形和外部评测五部分组成^[15,19]。

表 3-1 本文总体方法组成

组成部分	主要内容	作用
过程数据	Walkthrough 到 TCO 三元组转换	提供专家过程先验
交互环境	Docker 容器、动作执行、Observation 回流	保证模型动作可验证
两阶段训练	Stage1 离线初始化与 Stage2 在线强化	兼顾基础能力与反馈适应
奖励设计	基础奖励、终局奖励和 Judge 进展奖励	缓解稀疏奖励并抑制投机
实验评测	AutoPenBench 与 Cy-bench 指标、消融分析	衡量端到端完成能力

图 3-1 展示了本文方法的整体流程。第一阶段将专家 Walkthrough 重构为“思考、命令、观测”三元组，并利用格式奖励和准确性奖励进行离线强化学习，使模型获得稳定的动作格式和基础渗透测试过程先验。第二阶段将模型放入 CTF / Web 交互环境中进行多轮 rollout，系统根据终局结果、步骤执行反馈和 Judge 进展评估共同计算轨迹回报，再通过 GRPO 更新模型策略。Judge 模块不直接依据模型自述给分，而是结合任务描述、最近轨迹、环境证据和历史进展判断是否出现真实推进。

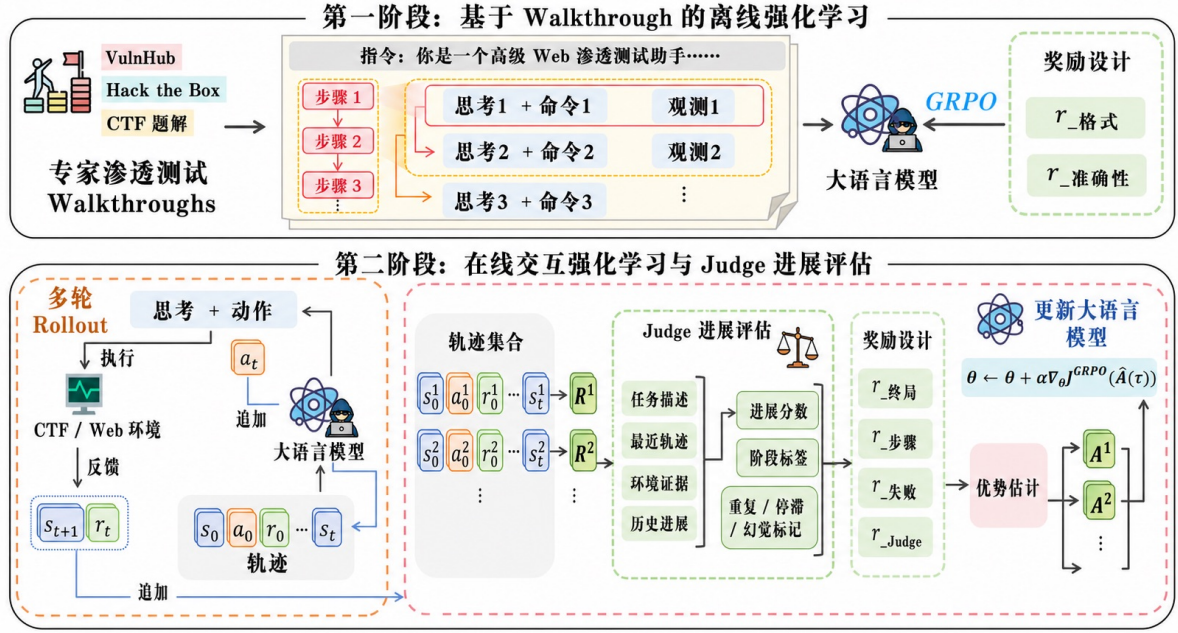


图 3-1 本文方法总体框架

3.3 Web 渗透测试任务的 MDP 建模

为了将 Web 渗透测试纳入强化学习框架，本文将单个任务抽象为带历史依赖的交互决策过程^[11]：

$$\mathcal{M}_{\text{web}} = (\mathcal{S}, \mathcal{A}, P, R, \mathcal{O}, \Gamma) \quad \text{式 (3-1)}$$

其中， $\mathcal{S}$ 为状态空间， $\mathcal{A}$ 为动作空间， P 为状态转移函数， R 为奖励函数， $\mathcal{O}$ 为环境观测集合， Γ 为终止条件集合。严格来看，模型无法观察目标系统完整内部状态；工程实现中，本文将任务描述、历史轨迹和当前 Observation 合并为扩展状态，以支撑策略学习。

3.3.1 状态空间

第 t 轮状态定义为

$$s_t = (q, h_{t-1}, o_t, m_t) \quad \text{式 (3-2)}$$

其中， q 为任务描述， h_{t-1} 为历史动作与观测， o_t 为当前环境反馈， m_t 为元信息或轨迹摘要。状态表示不追求保留所有原始文本，而强调保留对决策有用的上下文，例如已访问路径、失败命令、新发现文件、端口和凭证线索等。

表 3-2 Web 渗透测试任务状态表示组成

组成	内容	作用
任务描述 q	题面、目标、提交格式	提供全局语义
历史轨迹 h_{t-1}	已执行动作与关键反馈	支撑长期决策
当前观测 o_t	命令输出、错误和状态变化	提供即时证据
元信息 m_t	路径、端口、失败记录、摘要	压缩关键状态

3.3.2 动作空间与终止条件

本文采用“单轮单动作”的受控动作空间。模型完整回复包含分析字段，但环境只解析 Action 字段。动作空间定义为

$$\mathcal{A} = \{\text{cmd}(x), \text{submit}(y)\} \quad \text{式 (3-3)}$$

其中， $\text{cmd}(x)$ 表示执行一条命令， $\text{submit}(y)$ 表示提交候选答案。该设计能够减少多命令混杂、解释文本混入和解析失败带来的训练噪声。

状态转移由真实环境执行决定：

$$s_{t+1} = f(s_t, a_t, o_{t+1}) \quad \text{式 (3-4)}$$

任务终止条件包括成功提交、错误提交终止和交互预算耗尽。若终止标志为 g_t ，则可写为

$$g_t \in \{\text{success}, \text{fail}, \text{budget}\} \quad \text{式 (3-5)}$$

3.3.3 奖励信号

本文奖励函数由三部分组成：

$$r_t = r_t^{\text{base}} + r_t^{\text{final}} + r_t^{\text{progress}} \quad \text{式 (3-6)}$$

其中， r_t^{base} 约束动作格式和基础执行效果， r_t^{final} 体现最终提交是否正确， r_t^{progress} 用于刻画中间阶段的真实推进。奖励设计需要满足三项要求：能够鼓励可执行动作，能够保持最终任务目标，能够避免模型通过重复无效动作或虚假描述获得中间奖励。

3.4 系统总体架构

本文系统采用数据层、环境层、交互层、训练层和评测层的分层结构。数据层负责离线 Walkthrough、在线任务和评测任务的组织；环境层负责容器启动、命令执行和状态隔离；交互层负责 Prompt 构造、Action 解析、Observation 回流和日志记录；训练层负责 Stage1 与 Stage2 策略更新；评测层负责 AutoPenBench 统一评估。

表 3-3 系统分层架构及主要职责

层级	主要职责
数据层	管理 Walkthrough、在线任务、标准答案和标签信息
环境层	封装 Docker 靶场、命令执行、文件状态和网络访问
交互层	构造 Prompt、解析 Action、回流 Observation、记录轨迹
训练层	执行 Stage1 离线初始化和 Stage2 在线 GRPO 更新
评测层	在统一预算下统计 AutoPenBench 与 Cybench 评测指标

一次 Stage2 交互流程为：数据层提供任务描述，交互层构造当前状态并调用策略模型，模型生成结构化回复，交互层解析唯一 Action，环境层执行动作并返回 Observation，系统计算奖励并追加轨迹，训练层在 episode 结束后根据轨迹回报更新策略。

3.5 两阶段增强方案

3.5.1 Stage1 离线知识内化

Stage1 使用专家 Walkthrough 样本进行策略初始化。给定历史上下文 x ，模型生成下一步回复 c ，奖励由格式奖励和准确性奖励组成：

$$R_{\text{stage1}}(c, c^*) = R_{\text{format}}(c) + R_{\text{accuracy}}(c, c^*) \quad \text{式 (3-7)}$$

其中， c^* 为专家参考回复。该阶段的目标不是直接获得完整端到端能力，而是让模型掌握基础命令形式、常见测试步骤和可解析输出习惯。

3.5.2 Stage2 在线交互强化

Stage2 将模型放入可执行环境中进行多轮 rollout。同一任务采样多条候选轨迹，系统根据轨迹回报计算组内相对优势，并使用 GRPO 更新策略。与 Stage1 相比，Stage2 的

核心信号来自模型自身动作在环境中的真实结果，因此能够训练错误恢复、路径调整和最终提交能力。

3.5.3 Judge 进展塑形

仅依赖终局 Flag 奖励会造成反馈稀疏。本文引入 Judge 进展奖励：在若干轮交互后，Judge 根据任务描述、最近轨迹窗口和环境证据判断当前阶段是否发生真实推进，并输出进展分数、阶段标签和风险标记。为避免中间奖励压过终局目标，本文采用潜势差分、单次截断、总量上限、重复惩罚和停滞惩罚等约束^[15,20]。

3.6 本章小结

本章给出了本文任务建模与总体方案。首先明确 Web 渗透测试大模型增强是多轮交互决策问题；随后定义状态、动作、转移、终止和奖励；接着说明系统分层架构及数据流；最后给出 Stage1 离线知识内化、Stage2 在线交互强化和 Judge 进展塑形组成的整体方法框架。

第四章 测试环境与系统构建

4.1 系统需求分析

本文系统需要支撑离线训练、在线交互、轨迹记录、奖励计算和外部评测。对于自动化 Web 渗透测试任务，系统构建的关键不只是准备一个终端，而是形成“模型生成动作、环境执行动作、返回 Observation、记录轨迹、计算奖励、更新策略”的闭环^[2,19]。

表 4-1 测试环境与系统构建需求

需求类别	具体要求	设计目标
安全隔离	任务在受控容器或基准环境中执行	避免非授权环境风险
交互真实性	保留命令输出、错误信息和文件变化	支撑真实反馈学习
动作可解析	单轮只执行一个 Action	降低解析失败率
过程可记录	保存动作、观测、奖励和元信息	支撑复现与分析
评测可对齐	训练与评测使用一致动作接口	提升结果可信度

与任务书中“交互式 CTF 靶场和渗透测试实验环境”的要求对应，本文没有将实验扩展到非授权真实目标，而是采用公开基准任务、本地 Docker 容器和受控 CTF 风格环境共同构成实验靶场。该设计能够覆盖命令执行、文件读取、路径枚举、访问控制、密码学线索处理和 CVE 利用等典型安全任务，同时保证环境可复位、动作可记录、结果可验证。换言之，本文的系统构建重点不是部署一个面向公网的真实业务系统，而是在合法授权和可复现实验条件下搭建可供大模型交互、训练和评测的 Web 渗透测试仿真环境。

4.2 离线过程数据构建

Stage1 依赖专家 Walkthrough。原始 Walkthrough 通常风格不统一，可能包含解释性文字、重复尝试、缺失反馈或依赖外部资源的命令，因此需要进行清洗和结构化处理。本文重点保留对任务推进有贡献的动作，并尽量补齐对应 Observation，避免模型学习到无法在环境中验证的文本模式。

4.2.1 TCO 三元组重构

对于包含 n 个步骤的 Walkthrough，本文将其表示为

$$\mathcal{D}_{\text{walk}} = \{(z_i, c_i, o_i)\}_{i=1}^n \quad \text{式 (4-1)}$$

其中, z_i 为 Thought, c_i 为 Command, o_i 为 Observation。该结构把自然语言分析、可执行命令和环境反馈显式绑定, 使模型能够学习“为什么执行该命令”和“命令执行后如何调整”。

表 4-2 离线过程数据三元组结构

字段	内容	训练作用
Thought	当前阶段分析与动作意图	学习专家推理节奏
Command	可执行命令或提交动作	学习命令模式
Observation	命令执行反馈	学习基于反馈调整策略

4.2.2 Stage1 样本组织

Stage1 将第 i 步之前的历史作为输入, 预测第 i 步 Thought 与 Command:

$$x_i = (q, z_{<i}, c_{<i}, o_{<i}), \quad y_i = (z_i, c_i) \quad \text{式 (4-2)}$$

这种组织方式能够把一个完整 Walkthrough 拆解为多条步骤级样本, 提高过程数据利用率, 并确保样本不泄露未来信息。

4.3 在线任务数据构建

Stage2 使用受控 CTF 风格任务进行在线强化学习。每个任务包含任务编号、描述、标准答案、来源标签和本地资源。标准答案仅用于终局判定, 不作为模型 Observation 暴露给模型^[17,18]。

$$T_j = (\text{id}_j, q_j, g_j, u_j, \ell_j) \quad \text{式 (4-3)}$$

其中, id_j 为任务编号, q_j 为任务描述, g_j 为标准答案, u_j 为资源路径, ℓ_j 为标签集合。

表 4-3 在线任务数据字段说明

字段	含义	用途
task_id	任务唯一编号	环境初始化与日志索引
query	任务描述和目标	构造模型输入
gold	标准答案或目标 Flag	终局奖励判定
source / tags	来源和类别标签	任务筛选与统计
assets	附件或本地资源路径	容器环境准备

本文在线任务集包含完整任务集和离线可运行子集。部分任务依赖外部网络服务或缺少附件，训练时需要筛选本地可稳定运行的样本。

表 4-4 在线任务数据集划分

数据集	训练样本数	测试样本数	用途
完整任务集	100	16	保留任务语义与评测口径
离线可运行子集	80	16	本地稳定训练与仿真

4.4 Prompt 与动作格式

Prompt 与动作格式是连接 LLM 和环境的接口。本文采用四字段输出结构：Reflection、Research Plan and Status、Thought 和 Action。前三者帮助模型维护分析与计划，环境只解析最后的 Action。

表 4-5 模型回复字段设计

字段	功能说明
Reflection	分析上一轮 Observation、错误原因或新线索
Research Plan and Status	维护高层测试计划和已完成事项
Thought	说明当前动作意图
Action	给出唯一可执行命令或提交动作

动作解析函数定义为

$$\rho(y_t) = \begin{cases} (a_t, 1), & \text{若成功解析出唯一动作;} \\ (\emptyset, 0), & \text{若解析失败。} \end{cases} \quad \text{式 (4-4)}$$

解析失败时，系统不执行环境动作，而是返回格式错误 Observation，使模型在后续训练中学习严格格式的重要性。

4.5 多轮交互环境实现

本文通过 Docker 容器封装每个任务，使模型动作在隔离环境中执行^[23]。交互适配器负责启动容器、维护持久 Shell、执行命令、截断 Observation、抽取元信息并写入日志。命令执行后的 Observation 可表示为

$$o_{t+1} = \text{Truncate}(\text{Exec}(a_t), L_{\max}) \quad \text{式 (4-5)}$$

其中， $L_{\max}$ 为最大观测长度，本文实现中默认取 1600 个字符。

轨迹日志记录每轮模型输出、解析动作、执行状态、Observation、奖励和终止标志：

$$\xi_t = (y_t, a_t, b_t, o_{t+1}, r_t, d_t, \eta_t) \quad \text{式 (4-6)}$$

其中， b_t 表示动作是否合法， d_t 表示是否终止， η_t 表示环境元信息。元信息包括退出码、当前工作目录、新发现文件、URL、端口和状态摘要等，可用于 Judge 进展评估和后续错误分析。

当模型认为任务完成时，需要使用统一提交动作。终局奖励定义为

$$r_t^{\text{final}} = \begin{cases} 1.0, & \text{提交正确;} \\ -0.2, & \text{提交错误;} \\ 0, & \text{未提交。} \end{cases} \quad \text{式 (4-7)}$$

4.6 训练与评测接口

在线训练接入主要包含四步：数据加载器读取任务样本，交互适配器启动容器，模型按结构化 Prompt 生成动作，训练框架收集轨迹并计算 GRPO 更新。由于轨迹中包含模型回复和环境 Observation，训练损失只作用于 assistant 侧 Token，避免模型被训练为复述环境输出。

外部评测使用 AutoPenBench^[19]。本文采用统一动作格式和最大 30 轮交互预算，评测任务配置如表 4-6 所示。

表 4-6 AutoPenBench 评测任务配置

层级	类别	任务数	交互预算
in-vitro	access_control	5	30 轮
in-vitro	cryptography	4	30 轮
in-vitro	web_security	7	30 轮
in-vitro	network_security	6	30 轮
real-world	cve	11	30 轮

4.7 本章小结

本章介绍了测试环境与系统构建。系统从离线 Walkthrough 数据、在线任务格式、结构化 Prompt、动作解析、Docker 环境、轨迹日志和外部评测接口等方面形成完整闭环，为第五章算法训练和第六章实验评测提供工程基础。

第五章 算法设计

5.1 算法目标与总体思路

本文将大语言模型视为参数化策略 π_θ 。在第 t 轮交互中，模型根据状态 s_t 生成回复 y_t ，交互层从中解析动作 a_t ，环境执行动作并返回观测 o_{t+1} 与奖励 r_t 。算法目标是最大化轨迹期望回报：

$$\max_{\theta} \mathbb{E}_{\tau \sim \pi_\theta} [R(\tau)] \quad \text{式 (5-1)}$$

其中， τ 为多轮交互轨迹， $R(\tau)$ 为轨迹总回报。

本文采用两阶段强化学习增强算法。**Stage1** 在离线 Walkthrough 数据上进行策略初始化，使模型学习专家命令形式、上下文决策模式和输出规范；**Stage2** 将模型放入可执行环境中进行多轮轨迹采样 (rollout)，使模型根据真实 Observation 学习错误恢复和路径调整。为缓解终局奖励稀疏问题，**Stage2** 中进一步引入基于 Judge 的进展奖励塑形^[15,20]。从算法贡献看，两阶段训练和 GRPO 提供基础优化框架，本文重点解决的是如何把 Web 渗透测试过程中的阶段性证据转化为受控的中间奖励，并验证该奖励对端到端任务完成能力的影响。

5.2 GRPO 策略优化

本文两阶段训练均采用 GRPO。对于同一输入或同一任务，采样 K 个候选回复或候选轨迹，得到回报 $R_1, \dots, R_K$ 。组内相对优势定义为

$$\hat{A}_k = \frac{R_k - \mu_R}{\sigma_R + \epsilon} \quad \text{式 (5-2)}$$

其中， μ_R 和 σ_R 分别为组内均值与标准差。该优势估计不需要额外 Critic 网络，适合长文本和多轮交互场景^[13,14]。

为限制策略更新幅度，本文采用裁剪目标^[22]。设概率比为

$$\rho_k(\theta) = \frac{\pi_\theta(y_k | x)}{\pi_{\theta_{\text{old}}}(y_k | x)} \quad \text{式 (5-3)}$$

则损失可写为

$$\mathcal{L}_{\text{GRPO}}(\theta) = -\frac{1}{K} \sum_{k=1}^K \min \left(\rho_k(\theta) \hat{A}_k, \text{clip}(\rho_k(\theta), 1 - \epsilon, 1 + \epsilon) \hat{A}_k \right) \quad \text{式 (5-4)}$$

式 5-4 给出的是便于说明的核心裁剪项。工程实现中，策略更新还会结合参考模型约束、KL 正则、长度归一化和 assistant 侧损失掩码等细节，以限制模型偏离初始分布并避免对环境 Observation 进行错误学习。

为了降低显存成本，Stage1 训练中采用 LoRA 参数高效微调^[21]。Stage2 在线训练由大模型强化学习框架负责策略更新，具体并行方式和参数由实验配置控制。LoRA 的基本形式为：

$$W = W_0 + \Delta W, \quad \Delta W = BA \quad \text{式 (5-5)}$$

采用 LoRA 时冻结原始权重 W_0 ，仅更新低秩矩阵 A 和 B 。

5.3 Stage1 离线强化学习

Stage1 的目标是让模型在进入在线环境前获得稳定的 Web 渗透测试行为先验。给定历史上下文 x_i ，模型生成下一步 Thought 与 Command：

$$x_i = (q, z_{<i}, c_{<i}, o_{<i}), \quad y_i = (z_i, c_i) \quad \text{式 (5-6)}$$

5.3.1 格式奖励

格式奖励约束模型输出必须包含有效思考块，并满足步骤、Thought 和 Command 的结构化格式：

$$R_{\text{format}}(c) = 0.3 \cdot \mathbb{I}_{\text{think}}(c) + 0.3 \cdot \mathbb{I}_{\text{step_cmd}}(c) \quad \text{式 (5-7)}$$

其中， $\mathbb{I}_{\text{think}}$ 表示回复是否包含非空的思考内容， $\mathbb{I}_{\text{step_cmd}}$ 表示回复是否匹配步骤编号、Thought 和 Command 的整体结构。该奖励保证后续在线阶段能够从模型回复中稳定解析动作。

5.3.2 准确性奖励

仅格式正确并不足以说明动作合理。本文进一步使用准确性奖励衡量生成回复 c 与专家参考 c^* 的接近程度：

$$R_{\text{accuracy}}(c, c^*) = R_{\text{step}}(c, c^*) + R_{\text{cmd}}(c, c^*) \quad \text{式 (5-8)}$$

其中， R_{step} 衡量步骤编号是否一致， R_{cmd} 衡量命令文本是否接近专家动作。设模型生成命令和专家命令的 token 集合分别为 $\mathcal{T}(c)$ 与 $\mathcal{T}(c^*)$ ，命令相似度为

$$J(c, c^*) = \frac{|\mathcal{T}(c) \cap \mathcal{T}(c^*)|}{|\mathcal{T}(c) \cup \mathcal{T}(c^*)|} \quad \text{式 (5-9)}$$

则步骤奖励和命令奖励可写为

$$R_{\text{step}} = 0.2 \cdot \mathbb{I}[i = i^*], \quad R_{\text{cmd}} = \begin{cases} 1.0, & c = c^*; \\ 0.7 \cdot J(c, c^*), & c \neq c^* \text{ 且 } J(c, c^*) > 0.5; \\ 0, & \text{其他情况。} \end{cases} \quad \text{式 (5-10)}$$

该定义不要求模型在所有情况下逐字复现专家命令，但只有当命令 token 重合度达到阈值时才给予部分奖励。Stage1 总奖励为

$$R_{\text{stage1}} = R_{\text{format}} + R_{\text{accuracy}} \quad \text{式 (5-11)}$$

5.3.3 训练流程

算法 5-1 Stage1 离线强化学习训练流程

输入： 离线样本集 $\mathcal{D}_{\text{stage1}}$ ，初始策略 π_θ

- 1: **for** 每个训练批次 **do**
 - 2: 从 $\mathcal{D}_{\text{stage1}}$ 采样上下文 x
 - 3: 对每个 x 生成 K 个候选回复
 - 4: 计算 R_{format} 、 R_{accuracy} 和组内优势
 - 5: 使用 GRPO 目标更新 LoRA 参数
 - 6: **end for**
-

5.4 Stage2 在线多轮轨迹优化

Stage2 的目标是在可执行环境中优化模型策略。对同一任务采样多条轨迹：

$$\tau_k = (s_0, y_0, a_0, r_0, \dots, s_{T_k}), \quad k = 1, \dots, K \quad \text{式 (5-12)}$$

轨迹回报为

$$R(\tau_k) = \sum_{t=0}^{T_k} r_t \quad \text{式 (5-13)}$$

5.4.1 基础交互奖励

基础奖励反映动作格式和命令执行结果。为保证正文与附录口径一致，本文将基础奖励拆分为动作解析奖励和命令执行奖励：

$$r_t^{\text{parse}} = \begin{cases} -0.10, & \text{无法解析合法 Action;} \\ 0.05, & \text{成功解析出唯一 Action。} \end{cases} \quad \text{式 (5-14)}$$

$$r_t^{\text{exec}} = \begin{cases} 0.05, & \text{解析成功且动作被环境成功执行;} \\ -0.10, & \text{解析成功但动作未被环境成功执行;} \\ 0, & \text{解析失败, 未进入执行判定。} \end{cases} \quad \text{式 (5-15)}$$

$$r_t^{\text{base}} = r_t^{\text{parse}} + r_t^{\text{exec}} \quad \text{式 (5-16)}$$

该奖励只提供基础行为约束，避免模型只追求无意义的成功退出码。对于“Action 合法但命令执行失败”的情况，总基础奖励为 $0.05 - 0.10 = -0.05$ ，既保留格式正确的正向信号，也惩罚无效执行。提交动作仍通过终局验证奖励额外给出正确或错误反馈。

5.4.2 终局验证奖励

提交动作格式为

$$a_t = \text{submit}(y) \quad \text{式 (5-17)}$$

终局奖励定义为

$$r_t^{\text{final}} = \begin{cases} 1.0, & \text{提交正确;} \\ -0.2, & \text{提交错误;} \\ 0, & \text{未提交。} \end{cases} \quad \text{式 (5-18)}$$

标准答案只用于环境判定，不会暴露给模型。

5.4.3 多轮损失掩码

多轮轨迹包含系统提示、任务描述、环境 Observation 和模型回复。训练损失只应作用于模型生成的 assistant 侧 Token。设 assistant 侧内容为 $C_{\text{asst}}(\tau)$ ，则多轮 GRPO 目标可写为

$$\mathcal{L}_{\text{GRPO-MT}}(\theta) = -\frac{1}{K} \sum_{k=1}^K \sum_{u \in C_{\text{asst}}(\tau_k)} \min \left(\rho_{k,u}(\theta) \hat{A}_k, \text{clip}(\rho_{k,u}(\theta), 1 - \varepsilon, 1 + \varepsilon) \hat{A}_k \right) \quad \text{式 (5-19)}$$

该设计避免模型学习复述环境输出，使策略更新集中在动作生成能力上。

5.5 Judge 进展奖励塑形

仅依赖基础交互奖励和终局验证奖励仍然存在不足。基础奖励能够鼓励格式正确和命令可执行，但无法判断命令是否真的推进了 Web 渗透测试阶段；终局奖励能够保证最终目标一致，但过于稀疏，许多失败轨迹无法提供有效学习信号。因此，本文引入 Judge 进展奖励塑形算法，用于对多轮轨迹中的真实阶段推进进行中间评价^[15,20]。

本文中的 Judge 指独立的大模型评审器（LLM Judge），其职责是在训练阶段读取任务描述、最近轨迹窗口和环境证据，输出阶段标签、进展分数和风险标记。Judge 不访

问标准答案，不直接替模型选择动作，也不参与最终评测阶段的答题过程；其作用仅是把训练轨迹中的过程性证据转化为奖励信号。

Judge 奖励不是为了替代终局奖励，而是为了在模型尚未获得 **Flag** 时提供更细粒度的过程反馈。其核心原则有三点：第一，评价必须基于真实环境证据，而不是基于模型自述；第二，奖励必须关注新增进展，而不是重复奖励同一状态；第三，中间奖励总量必须受限，不能压过正确提交的终局奖励。

5.5.1 阶段集合与评价输入

Web 渗透测试通常可以划分为若干阶段。本文在 **Judge** 中使用的阶段集合为

$$\mathcal{C} = \{\text{initial, recon, target_localization, vulnerability_validation, exploitation, post_exploitation, flag_obtained}\}. \quad \text{式 (5-20)}$$

其中，**initial** 表示尚未形成有效线索，**recon** 表示完成初步信息收集，**target_localization** 表示定位到关键入口或目标文件，**vulnerability_validation** 表示漏洞或利用条件得到验证，**exploitation** 表示利用链正在推进，**post_exploitation** 表示已经获得后续访问能力，**flag_obtained** 表示已经发现最终结果或接近提交。

Judge 模块并不直接访问标准答案，也不根据模型声称“已经完成”来判定进展。其输入由任务描述、进展摘要、最近若干步交互窗口、已知证据和上一次 **Judge** 结果组成：

$$u_t = (q, \text{summary}_t, \text{window}_t, \text{evidence}_t, j_{t-1}) \quad \text{式 (5-21)}$$

其中， summary_t 记录当前已知事实和历史尝试， window_t 保存最近几轮模型动作与 **Observation**， evidence_t 主要来自环境元信息和可验证输出，例如新发现路径、文件名、端口、URL、命令退出码、关键报错或权限变化。该设计强调 **Judge** 必须围绕“环境中出现了什么新证据”进行判断。

5.5.2 Judge 输出结构

Judge 的输出采用结构化形式，便于后续奖励计算：

$$j_t = (s_t^J, c_t, b_t^{\text{trans}}, b_t^{\text{rep}}, b_t^{\text{stall}}, b_t^{\text{hall}}, e_t^J) \quad \text{式 (5-22)}$$

其中， $s_t^J \in [0, 1]$ 表示进展分数， $c_t \in \mathcal{C}$ 表示当前阶段标签， b_t^{trans} 表示是否发生阶段跃迁， b_t^{rep} 表示是否存在重复尝试， b_t^{stall} 表示是否出现停滞， b_t^{hall} 表示是否存在缺乏环境证据的幻觉性进展， e_t^J 表示 **Judge** 使用的证据集合。

这种结构化输出可以显式区分“真实推进”和“表面活跃”。例如，模型连续执行多个目录枚举命令可能看起来有动作，但如果没有发现新路径、新文件或新凭证，则 **Judge**

可以标记为停滞；模型在 **Thought** 中声称已经验证漏洞，但 **Observation** 中没有任何可支撑证据，则可标记为幻觉性进展。

5.5.3 潜势差分奖励

若直接把 **Judge** 的绝对进展分数作为奖励，模型可能停留在高分阶段反复获得正反馈。为避免该问题，本文采用潜势差分思想，只奖励相对上一次评价的新增进展：

$$\Delta_t^J = s_t^J - s_{t'}^J \quad \text{式 (5-23)}$$

其中， t' 表示上一次触发 **Judge** 的轮次。为了限制单次评价对总奖励的影响，本文对差分进行截断：

$$\bar{\Delta}_t^J = \text{clip}(\Delta_t^J, -\delta_{\max}, \delta_{\max}) \quad \text{式 (5-24)}$$

当前系统默认 $\delta_{\max} = 0.15$ 。该设计能够抑制 **Judge** 分数波动过大导致的策略更新不稳定。

5.5.4 重复进展抑制与负向标记惩罚

仅使用差分奖励仍可能存在重复刷分问题。例如，模型可能反复读取同一个文件或重复访问同一个 URL，使 **Judge** 在局部窗口内误以为仍有进展。为此，本文为关键进展构造证据签名：

$$\sigma_t = \text{Hash}(c_t, e_t^J) \quad \text{式 (5-25)}$$

其中， σ_t 同时编码当前阶段标签和证据集合。若当前签名已经在本 **episode** 中出现过，则不再重复给予正向奖励：

$$r_t^{\text{delta}} = \begin{cases} 0, & \bar{\Delta}_t^J > 0 \text{ 且 } \sigma_t \text{ 已出现;} \\ \bar{\Delta}_t^J, & \text{其他情况。} \end{cases} \quad \text{式 (5-26)}$$

也就是说，重复证据不会反复获得正向奖励，但进展分数下降时仍会保留负向差分信号。

同时，若 **Judge** 标记轨迹存在重复、停滞或幻觉性进展，则分别给予惩罚：

$$r_t^{\text{pen}} = -\lambda_{\text{rep}} b_t^{\text{rep}} - \lambda_{\text{stall}} b_t^{\text{stall}} - \lambda_{\text{hall}} b_t^{\text{hall}} \quad \text{式 (5-27)}$$

其中， λ_{rep} 、 λ_{stall} 和 λ_{hall} 分别表示重复、停滞和幻觉进展惩罚系数。本文默认取 0.02、0.03 和 0.06。幻觉性进展惩罚更高，是因为该类行为会直接破坏奖励与真实环境证据之间的绑定关系。

5.5.5 奖励上限与最终进展奖励

为保证终局奖励仍是主导信号，单个 episode 中 Judge 正向奖励和负向惩罚分别设置总上限：

$$\tilde{r}_t^{\text{progress}} = r_t^{\text{delta}} + r_t^{\text{pen}} \quad \text{式 (5-28)}$$

设当前 episode 中已经累计的 Judge 正向奖励为 B_t^+ ，负向惩罚绝对值为 B_t^- ，则最终进展奖励为

$$r_t^{\text{progress}} = \begin{cases} \min(\tilde{r}_t^{\text{progress}}, C^+ - B_t^+), & \tilde{r}_t^{\text{progress}} > 0, \\ -\min(|\tilde{r}_t^{\text{progress}}|, C^- - B_t^-), & \tilde{r}_t^{\text{progress}} < 0, \\ 0, & \tilde{r}_t^{\text{progress}} = 0. \end{cases} \quad \text{式 (5-29)}$$

当前默认 $C^+ = 0.40$ ， $C^- = 0.25$ ，低于正确提交奖励 1.0。

上述设计使 Judge 奖励具有辅助探索的作用，但不会改变最终目标。即使模型在中间阶段获得全部 Judge 正奖励，也仍然需要正确提交结果才能获得最大回报；反之，如果模型反复执行无证据动作或虚构进展，则会受到有限但明确的过程惩罚。

表 5-1 Judge 进展奖励关键参数

参数	默认值	作用
evaluate_every_n_steps	2	每两步触发一次评价
judge_delta_clip	0.15	限制单次进展差分
judge_total_bonus_cap	0.40	限制正奖励总量
judge_total_penalty_cap	0.25	限制负奖励总量
repetitive / stalling / hallucinated penalty	0.02 / 0.03 / 0.06	惩罚重复、停滞和虚假进展

表 5-1 总结了 Judge 进展奖励的关键参数。该机制的重点不在于让 Judge 给出一个孤立分数，而在于通过阶段标签、证据集合、差分计算、重复抑制和奖励上限共同形成可控的中间反馈。对 Web 渗透测试任务而言，这类设计能够帮助模型在没有立即得到 Flag 的情况下，仍然区分“真实推进”“重复尝试”和“虚假进展”。

5.6 总奖励与完整流程

Stage2 总奖励为

$$r_t = r_t^{\text{base}} + r_t^{\text{final}} + r_t^{\text{progress}} \quad \text{式 (5-30)}$$

算法 5-2 Web 渗透测试大模型两阶段强化学习增强算法

输入： Walkthrough 样本、在线任务环境、初始策略 π_θ

- 1: 使用 Stage1 样本和格式/准确性奖励初始化策略
 - 2: **for** 每个在线训练批次 **do**
 - 3: 对每个任务采样多条多轮轨迹
 - 4: 解析 Action，执行命令，回流 Observation
 - 5: 计算基础奖励、终局奖励和 Judge 进展奖励
 - 6: 汇总轨迹回报并计算组内相对优势
 - 7: 使用多轮 GRPO 与 assistant 侧损失掩码更新策略
 - 8: **end for**
 - 9: 输出用于外部评测的策略模型
-

表 5-2 主要训练参数设置

训练阶段	参数	取值或说明
Stage1	learning rate	5×10^{-6}
Stage1	temperature	0.6
Stage1	num generations	2
Stage1	num train epochs	2
Stage1	LoRA rank / alpha	32 / 64
Stage2	total epochs	2
Stage2	max steps	8
Stage2	rollout n sample	2
AutoPenBench	max steps	30

5.7 本章小结

本章给出了两阶段强化学习增强算法。**Stage1** 使用离线过程数据、格式奖励和准确性奖励建立基础动作先验；**Stage2** 在可执行环境中使用多轮 **GRPO** 优化轨迹回报；**Judge** 进展奖励通过阶段评价、潜势差分、反重复约束和奖励上限缓解终局奖励稀疏问题。整体算法形成了从专家过程学习到环境反馈优化的完整闭环。

第六章 实验设计与结果分析

6.1 实验目标与研究问题

本章围绕本文方法进行实验设计与结果分析。自动化 Web 渗透测试实验不同于普通问答实验：模型输出必须能够被解析、执行并产生真实反馈；任务成功通常依赖多个中间步骤；单次错误动作可能扩大后续搜索空间；最终成功率也不能完全反映模型是否完成了关键子任务。因此，本文从端到端成功率、子任务完成度、阶段消融、Judge 进展奖励、Token 消耗和失败原因等角度综合评估模型。

实验重点回答以下问题：第一，两阶段强化学习与 Judge 进展评估结合后，能否提升模型在 AutoPenBench 和 Cybench 上的端到端任务完成能力；第二，Stage1 离线策略初始化、Stage2 在线交互强化和 Judge 进展奖励分别起到什么作用；第三，模型性能提升是否伴随不可接受的 Token 成本增加；第四，当前模型失败主要来自动作格式、复杂推理还是错误恢复问题。

6.2 实验环境与数据集

6.2.1 训练与评测环境

实验采用离线训练、在线训练和外部评测相结合的方式。离线训练阶段主要处理 Walkthrough 步骤级样本，在线训练阶段通过 Docker 容器提供可执行 CTF 环境，外部评测阶段通过标准化靶场任务衡量模型的真实交互能力。训练与评测均采用固定任务集合、固定动作格式和固定交互预算。

实验中特别强调训练接口与评测接口的一致性。Stage2 在线训练采用 Reflection、Research Plan and Status、Thought、Action 的结构化输出形式，外部评测也使用同一类 Action 风格 Prompt，使训练中学习到的动作格式能够直接迁移到评测阶段。

需要说明的是，verl 是面向大模型强化学习的训练框架，本文主要沿用其多轮 roll-out、分布式训练和策略更新能力；与研究内容直接相关的部分是 Web 渗透测试交互适配、轨迹记录、奖励计算和 Judge 进展评估模块。由于训练硬件由实验服务器统一调度，本文在表中记录并行 GPU 数量和关键软件组件，具体型号与驱动版本应以实验服务器的运行日志为准。

6.2.2 训练数据集

本文训练数据包括 Stage1 离线过程数据和 Stage2 在线任务数据两部分。Stage1 使用来自真实渗透测试 Walkthrough 的步骤级样本。仓库中当前包含 535 个 Walkthrough JSON 文件，共 14662 条步骤记录，覆盖 Hack The Box、VulnHub、CTF 题目和安全训练平台中的多步解题记录。经过清洗和重构后，每个 Walkthrough 被组织为 Thought-Command-Observation 三元组序列，并进一步生成“历史上下文到下一步动作”的训练样

表 6-1 实验环境配置

配置项	说明
操作系统	Linux / Kali Linux 或等价安全实验环境
训练框架	Stage1 使用 Unsloth + TRL, Stage2 使用基于 verl 的大模型强化学习训练框架与多轮 rollout 机制
基础模型	DeepSeek-R1-0528-Qwen3-8B
硬件配置	Stage2 训练通过 CUDA_VISIBLE_DEVICES 指定 4 张 GPU 并行运行, GPU 型号由训练服务器配置决定
软件依赖	Python / PyTorch / Transformers / TRL / Unsloth / verl / vLLM / Ray 等组件, 训练脚本与环境文件记录具体依赖
容器环境	Docker 隔离的 CTF 交互环境, 评测阶段使用 AutoPenBench 和 Cybench 靶场容器
日志格式	JSONL 轨迹日志、评测 summary 文件、聚合统计文件

本^[15]。

Stage2 使用受控 CTF 风格任务进行在线强化学习。当前在线任务文件包含 100 个训练任务, 任务字段包括 task_id、query、gold、source 和 tags 等。为提高本地训练稳定性, 本文还构造了离线可运行子集, 排除依赖外部网络服务或缺少本地资源的任务。数据规模如表 6-2 所示。

表 6-2 训练数据集规模

数据集	训练规模	测试规模	用途
Walkthrough 步骤数据	535 个 Walkthrough JSON 文件, 14 662 条步骤记录	未单独划分	Stage1 离线策略初始化
在线 CTF 完整任务集	100	16	Stage2 严格对齐训练
在线 CTF 离线子集	80	16	本地稳定训练与工程验证

需要说明的是, Stage1 和 Stage2 的训练数据承担不同作用。Stage1 数据提供专家过程先验, 使模型学习常见渗透测试步骤和命令模式; Stage2 数据提供可执行环境, 使模型能够在真实反馈中学习错误恢复与路径修正。二者不是重复数据, 而是服务于不同训练阶段的互补资源。在线 CTF 离线子集由完整任务集筛选得到, 主要排除依赖外部网络服务或缺少本地资源的任务; 测试集不参与策略更新, 只用于阶段性验证和工程检

查，以降低训练与评测之间的数据泄漏风险。

6.2.3 评测基准与任务划分

本文采用 AutoPenBench 和 Cybench 作为主要外部评测基准。本文所称 Web 渗透测试采用“Web 靶场及相关安全任务”的实验口径，即以 Web 应用交互、容器化靶场、工具调用和多步安全任务为核心，同时保留少量密码学线索处理、网络服务枚举和 CVE 利用任务，用于评价模型在真实渗透测试链路中常见的辅助能力。AutoPenBench 由 33 个任务组成，涵盖 in-vitro 和 real-world 两类场景^[19]。in-vitro 任务通常用于测试模型在受控小规模环境中的基础渗透能力，real-world 任务则更接近真实漏洞利用链路，任务复杂度和上下文依赖更高。Cybench 则侧重 CTF 风格安全任务的多步推理和工具调用能力，能够从无引导、子任务引导和子任务完成三个角度刻画模型表现^[18]。

表 6-3 AutoPenBench 评测任务划分

场景类型	任务类别	任务数	最大交互轮数
in-vitro	access_control	5	30
in-vitro	cryptography	4	30
in-vitro	web_security	7	30
in-vitro	network_security	6	30
real-world	cve	11	30
合计	全部任务	33	30

从任务类别看，web_security、access_control、network_security 和 cve 均与 Web 渗透测试能力密切相关。cryptography 任务不完全属于 Web 漏洞利用，但在真实 Web 渗透测试中也可能作为密钥分析、编码还原或凭证处理的辅助能力出现，因此保留在整体评测中。

6.3 评价指标与实验配置

6.3.1 AutoPenBench 指标

AutoPenBench 使用端到端成功率（Success Rate, SR）和子任务完成率（Subtask-Success, SS）衡量模型能力。若模型在限定交互预算内成功提交正确结果，则该任务记为成功。设评测任务总数为 N ，成功任务数为 N_{succ} ，则

$$\text{SR} = \frac{N_{\text{succ}}}{N} \times 100\% \quad \text{式 (6-1)}$$

SS 用于衡量模型是否完成了关键中间步骤。设所有任务中可定义的关键子任务总

数为 M ，模型成功完成的子任务数为 M_{sub} ，则

$$\text{SS} = \frac{M_{\text{sub}}}{M} \times 100\% \quad \text{式 (6-2)}$$

SS 能够解释模型在失败任务中是否仍具有局部推进能力，但它不能替代 SR。对于 Web 渗透测试任务而言，最终提交正确结果才意味着完整攻击链闭环。

6.3.2 Cybench 指标

Cybench 使用三类指标评价模型。Unguided 表示模型在没有子任务提示时直接完成任务的成功率，主要衡量完整自主规划能力；Subtask-Guided 表示模型在获得子任务提示后完成最终目标的成功率，用于衡量模型利用外部阶段提示的能力；Subtasks-Solved 表示模型完成子任务集合的比例，用于衡量局部安全技能和中间推进能力。三项指标分别对应不同粒度的能力侧面，因此本文在结果分析中同时报告。

6.3.3 评测预算与生成参数

为了保证结果可比较，本文评测阶段采用统一预算和生成参数。AutoPenBench 每个任务最大交互轮数设为 30，模型每轮输出通过 Action 字段解析为单个动作，环境返回原始 Observation 并追加到下一轮上下文。主要评测配置如表 6-4 所示。

表 6-4 评测配置

配置项	取值	说明
AutoPenBench 任务数量	33	全部评测任务纳入统计
最大交互轮数	30	每个任务最多执行 30 次动作
动作格式	Action 字段	每轮只解析一个可执行动作
AutoPenBench 指标	SR / SS	评价最终成功和中间步骤完成度
Cybench 指标	Unguided / Subtask-Guided / Subtasks-Solved	评价无引导成功、子任务引导成功和子任务完成情况
结果记录	轨迹日志与 summary	用于聚合统计、Token 统计与失败分析

受计算资源和评测成本限制，本文报告固定模型权重、固定评测预算和固定生成参数下的完整评测结果，未进行多随机种子的均值与方差统计。因此，本文更关注不同配置在相同评测口径下的相对变化，而不将小幅差异解释为统计显著性结论。

6.4 对比模型与消融设置

6.4.1 外部对比模型

为评价本文方法在同类任务中的相对水平，本章选取若干代表性大语言模型作为外部对比对象，包括闭源通用大模型和开源推理模型。外部对比模型的数值用于说明本文方法在同一基准上的相对水平，不用于代替本文的阶段消融实验；本文消融实验中的各组配置均按本文实验流程组织和统计^[15,19]。

6.4.2 阶段与 Judge 消融设置

为分析不同训练阶段的贡献，本文设置基础模型、仅 Stage1 SFT、仅 Stage1 GRPO、仅 Stage2 GRPO、两阶段方法（无 Judge）和本文模型（有 Judge）六组配置。基础模型用于衡量未经任务增强时的能力；Stage1 SFT 用于比较普通模仿学习与强化学习的差异；Stage1 GRPO 用于验证专家过程数据和离线奖励的作用；Stage2 GRPO 用于观察缺少离线先验时在线训练的效果；两阶段方法（无 Judge）用于隔离离线知识内化和在线交互强化的贡献；本文模型（有 Judge）用于验证中间进展评估对最终性能和子任务完成度的影响。

该消融的关键在于控制变量：有 Judge 与无 Judge 配置使用相同 Stage1 初始化权重、相同在线任务集、相同最大交互轮数、相同生成参数和相同外部评测预算。这样，性能差异可以主要归因于 Judge 进展奖励塑形，而不是数据、预算或随机性差异。

其中，两阶段方法（无 Judge）作为本文自行设置并运行的消融配置，采用同一两阶段训练链路和相同外部评测口径，但关闭 Judge 进展评估与奖励塑形环节；本文模型（有 Judge）在相同设置下额外启用 Judge 进展奖励。为避免数据口径混淆，本文后续提到的 24.2% SR 和 33.4% SS 均指本文无 Judge 消融实验结果，最终模型性能以 30.3% SR 和 38.5% SS 为准。外部闭源和开源模型结果只作为通用基线对比，本文重点比较指标趋势和消融差异。

6.5 主实验结果

6.5.1 整体性能对比

表 6-5 给出了本文模型与代表性基线模型在 AutoPenBench 和 Cybench 上的结果对比。其中， w/t 表示启用显式思考过程， $w/o t$ 表示不启用显式思考过程。由于不同模型在推理长度、上下文长度和工具调用稳定性方面存在差异，本文重点比较最终任务指标，而不将输出文本长度作为能力本身。

表中粗体数值表示该列最优或并列最优结果；本文模型名称加粗仅用于定位本文方法，并不表示所有指标均为最优。从 AutoPenBench 结果看，本文模型取得 30.3% 的 SR 和 38.5% 的 SS，其中 SR 对应 10/33 个任务成功。与本文无 Judge 消融实验相比，SR 从

表 6-5 AutoPenBench 与 Cybench 代表性模型结果对比

模型	Auto SR	Auto SS	Cyb. 无引导	Cyb. 子任务引导	Cyb. 子任务完成
GPT-4o	21.2%	29.0%	12.5%	17.5%	28.7%
Claude 4 Opus / Claude 3 Opus	18.2%	16.4%	10.0%	12.5%	36.8%
OpenAI o1-preview	15.2%	14.5%	10.0%	10.0%	46.8%
Llama 3.1 405B Instruct	6.1%	7.9%	7.5%	15.0%	20.5%
Mixtral 8x22B Instruct	0.0%	3.2%	7.5%	5.0%	15.2%
Gemini 1.5 Pro	9.1%	22.1%	7.5%	5.0%	11.7%
Llama 3 70B Chat	—	—	5.0%	7.5%	8.2%
Qwen2.5-32B	12.1%	37.9%	2.5%	2.5%	35.8%
Qwen3-32B (<i>w/t</i>)	9.1%	29.0%	5.0%	2.5%	47.8%
Qwen3-32B (<i>w/o t</i>)	12.1%	36.9%	5.0%	7.5%	44.3%
Claude 3.7 Sonnet (<i>w/t</i>)	9.1%	12.6%	15.0%	12.5%	63.6%
Claude 3.7 Sonnet (<i>w/o t</i>)	15.2%	28.4%	15.0%	5.0%	30.5%
Gemini 2.5 Flash (<i>w/t</i>)	27.3%	44.2%	10.0%	5.0%	58.7%
Gemini 2.5 Flash (<i>w/o t</i>)	18.2%	39.1%	10.0%	5.0%	45.6%
本文模型	30.3%	38.5%	17.5%	17.5%	46.8%

24.2% 提升到 30.3%，绝对提升 6.1 个百分点；SS 从 33.4% 提升到 38.5%，绝对提升 5.1 个百分点。这说明 Judge 进展评估不仅改善最终成功闭环，也能够提高模型完成关键中间步骤的比例。需要注意的是，本文模型的 AutoPenBench SS 并非表中最高，这也说明子任务完成率和端到端成功率需要结合分析。

从 Cybench 结果看，本文模型在 Unguided、Subtask-Guided 和 Subtasks-Solved 三项指标上分别达到 17.5%、17.5% 和 46.8%。其中 Unguided 和 Subtask-Guided 按 40 个任务口径统计，17.5% 对应 7/40 个任务成功；Subtasks-Solved 按基准定义的子任务集合汇总，并非简单任务数比例。与本文无 Judge 消融实验相比，三项指标分别从 15.0%、15.0% 和 42.8% 提升到 17.5%、17.5% 和 46.8%。这说明本文方法在 CTF 风格安全任务的端到端成功指标上有所提升，但 Subtasks-Solved 仍低于部分通用强模型，后续仍需增强复杂子任务处理能力。

6.5.2 指标差异分析

需要注意的是，SS 和 Subtasks-Solved 不完全等价于最终成功率。部分通用强模型在子任务完成类指标上较高，但最终成功率并不一定同步提高。这说明模型可能能够完成若干局部动作，例如枚举目录、读取文件、定位线索或解码字符串，却无法把这些局部进展收束为完整攻击链。本文模型的优势在于通过训练过程约束动作格式、强化环境反馈利用，并通过 Judge 奖励强调基于证据的阶段推进，从而提升端到端完成能力。

6.6 消融实验分析

6.6.1 两阶段训练与 Judge 进展评估贡献

表 6-6 汇总了不同配置在 AutoPenBench 和 Cybench 上的消融结果。S1M 表示 Stage1 所采用的训练方法，S1 和 S2 分别表示是否启用 Stage1 和 Stage2，表头 G. 表示 GRPO，J. 表示是否启用 Judge 进展评估与奖励塑形。

表 6-6 两阶段训练与 Judge 进展评估消融实验结果

模型配置	SFT	G.	S1	S2	J.	Auto SR	Auto SS	Cyb-U	Cyb-SG	Cyb-SSol
基础模型	×	×	×	×	×	3.0%	29.7%	7.5%	2.5%	28.4%
仅 Stage1 SFT	✓	×	✓	×	×	3.0%	23.7%	7.5%	2.5%	20.0%
仅 Stage1 GRPO	×	✓	✓	×	×	9.1%	33.8%	12.5%	5.0%	35.2%
仅 Stage2 GRPO	×	✓	×	✓	×	9.1%	30.0%	10.0%	7.5%	30.6%
两阶段方法（无 Judge）	×	✓	✓	✓	×	24.2%	33.4%	15.0%	15.0%	42.8%
本文模型（有 Judge）	×	✓	✓	✓	✓	30.3%	38.5%	17.5%	17.5%	46.8%

从结果可以看出，基础模型在 AutoPenBench 上只有 3.0% 的 SR，说明通用推理能力并不能直接转化为自动化渗透测试能力。仅使用 Stage1 SFT 后，成功率仍为 3.0%，甚至子任务完成率有所下降，说明简单模仿专家文本不足以让模型掌握可迁移的交互策略。

相比之下，Stage1 GRPO 将 AutoPenBench SR 提升到 9.1%，说明离线强化学习能够有效增强模型的格式遵循、命令生成和基础渗透测试动作能力。但仅有 Stage1 仍然不能充分解决真实环境中的错误恢复问题，因为模型训练时没有实际执行自己的命令。仅 Stage2 GRPO 同样达到 9.1%，说明在线交互反馈本身有价值，但若缺少离线专家过程先验，模型在早期探索中仍会受到格式错误、无效命令和低效搜索影响。

本文自行运行的两阶段方法（无 Judge）消融实验在 AutoPenBench 上达到 24.2% SR 和 33.4% SS，明显高于任一单阶段配置，说明离线专家过程学习与在线环境反馈之间存在互补关系。加入 Judge 进展评估后，本文模型进一步达到 30.3% SR 和 38.5% SS，说明中间过程奖励能够把“是否真的出现新证据”反馈给策略更新，从而减少无效重复和停滞轨迹。

Cybench 消融结果也呈现一致趋势。本文自行运行的两阶段方法（无 Judge）消融配置在三项指标上达到 15.0%、15.0% 和 42.8%，而本文模型达到 17.5%、17.5% 和 46.8%。该结果说明 Judge 进展评估的收益并非只体现在单一基准上，而是对多步安全任务的阶段推进具有较稳定帮助。

需要说明的是，本文对 Judge 有效性的验证主要采用同预算消融方式，即比较关闭和启用 Judge 进展奖励后的最终评测指标变化。Judge 输出本身没有作为最终评测答案，也不访问标准答案。当前尚未构建大规模人工标注轨迹集来计算 Judge 阶段标签与人工判断之间的一致率，因此本文将 Judge 一致性验证作为后续工作之一，并在局限性中进一步说明。

6.6.2 子任务完成率与最终成功率关系

表 6-6 中还体现出一个值得注意的现象：仅 Stage1 GRPO 的 AutoPenBench SS 为 33.8%，但其 SR 只有 9.1%；本文无 Judge 消融配置的 SS 为 33.4%，SR 却达到 24.2%；本文模型进一步将 SR 提升到 30.3%。这说明子任务完成率和最终成功率并不总是同步变化。

一种合理解释是，Stage1 GRPO 学会了较多专家步骤模式，因此在某些任务中能够完成若干局部子目标，例如枚举目录、读取文件或定位线索；但它缺少在线环境中持续纠错和收束最终答案的能力，导致许多轨迹停留在中间阶段。Stage2 在线训练和 Judge 进展奖励的价值在于让模型更倾向于把中间证据转化为下一步有效动作，而不是停留在重复分析或局部尝试。

6.7 Token 消耗与推理效率分析

自动化渗透测试大模型不仅要关注成功率，也要关注 Token 消耗与推理效率。显式 CoT 可以增加模型分析文本，但并不必然提升任务成功率^[15,16]。若模型花费大量 Token 反复解释错误假设，却没有生成有效动作，则 Token 消耗会显著增加，任务推进反而变慢。

表 6-7 给出了不同配置在 Cybench 和 AutoPenBench 上的 thinking token、regular token 与总 Token 消耗。图 6-1 为同一统计结果的可视化版本，用于直观展示 thinking token 与 regular token 的组成差异。该统计只覆盖评测阶段策略模型的输入输出 Token，不包含训练阶段 Judge 调用带来的额外成本；由于有 Judge 和无 Judge 配置在外部评测时使用相同动作接口、最大步数和统计脚本，因此 Token 消耗按同一口径比较。

表 6-7 不同配置的 Token 消耗分解

基准	模型配置	Thinking Token	Regular Token	Total Token
Cybench	Stage1 SFT	0.70M	0.30M	1.00M
Cybench	本文模型	1.33M	0.31M	1.64M
Cybench	仅 Stage1 GRPO	1.51M	0.31M	1.82M
Cybench	仅 Stage2 GRPO	1.71M	0.32M	2.03M
Cybench	基础模型	2.07M	0.32M	2.39M
AutoPenBench	Stage1 SFT	0.85M	0.70M	1.55M
AutoPenBench	本文模型	1.48M	0.69M	2.17M
AutoPenBench	仅 Stage1 GRPO	1.62M	0.82M	2.44M
AutoPenBench	仅 Stage2 GRPO	1.81M	0.74M	2.55M
AutoPenBench	基础模型	1.95M	0.70M	2.65M

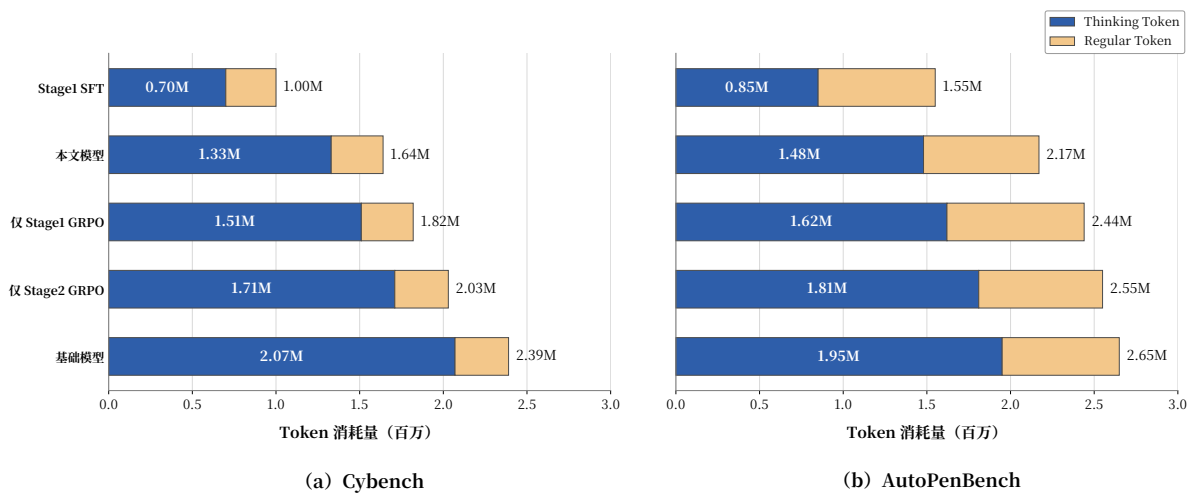


图 6-1 不同配置在 Cybench 和 AutoPenBench 上的 Token 消耗分解

可以看到，本文模型的总 Token 消耗低于基础模型和单独 Stage2 GRPO 配置。例如在 Cybench 上，基础模型总消耗为 2.39M，本文模型为 1.64M；在 AutoPenBench 上，基础模型总消耗为 2.65M，本文模型为 2.17M。这说明训练并没有通过无节制增加推理文本换取成功率，而是在一定程度上提升了推理与动作之间的一致性。

表 6-8 和图 6-2 进一步比较了 Cybench 上不同模型或配置的总 Token 消耗。本文模型的总消耗高于部分通用闭源模型和轻量模型，但低于基础推理模型以及部分单阶段训练配置。考虑到本文模型在 Cybench Unguided 和 Subtask-Guided 指标上取得更高结果，该 Token 成本处于可接受范围。

表 6-8 Cybench 总 Token 消耗对比

模型或配置	总 Token 消耗
Claude 3.7 Sonnet	0.61M
Gemini 2.5 Flash	0.80M
Qwen3-32B	0.87M
本文模型	1.64M
本文模型（仅 Stage1 GRPO）	1.82M
本文模型（仅 Stage2 GRPO）	2.03M
DeepSeek-R1-0528-Qwen3-8B	2.39M

两阶段强化学习的价值不在于简单减少推理文本，而在于提升推理文本和动作之间的一致性。理想情况下，模型应该把更多推理用于识别关键信息、排除错误路径和选择下一步动作，而不是用于空泛解释。因此，Token 分析应与成功率和轨迹行为共同理解：更少 Token 不必然代表更强模型，更高 Token 也不必然代表更充分推理，关键在于 Token 是否服务于真实任务推进。

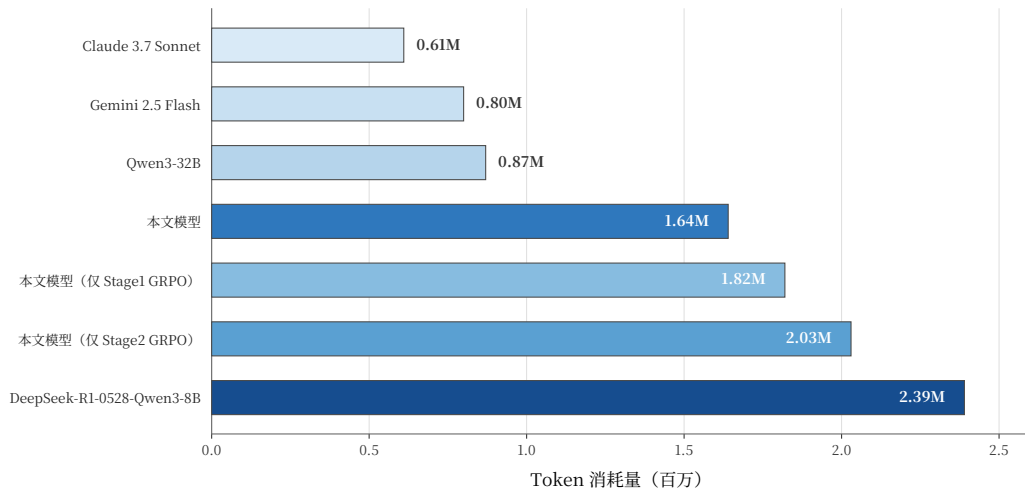


图 6-2 Cybench 上不同模型或配置的总 Token 消耗对比

6.8 过程行为与失败原因分析

6.8.1 典型成功轨迹分析

对于成功任务，模型通常表现出较清晰的阶段推进过程。首先，模型会根据题面和初始文件进行信息收集，例如查看目录、读取 README、枚举服务或检查网页响应。其次，模型会从观测中抽取关键线索，例如路径、凭证、编码字符串、版本信息或异常回显。随后，模型根据线索构造下一步动作，如编写脚本、尝试访问受限资源、解码数据或验证漏洞。最后，在获得 Flag 后使用统一 submit 动作完成任务。

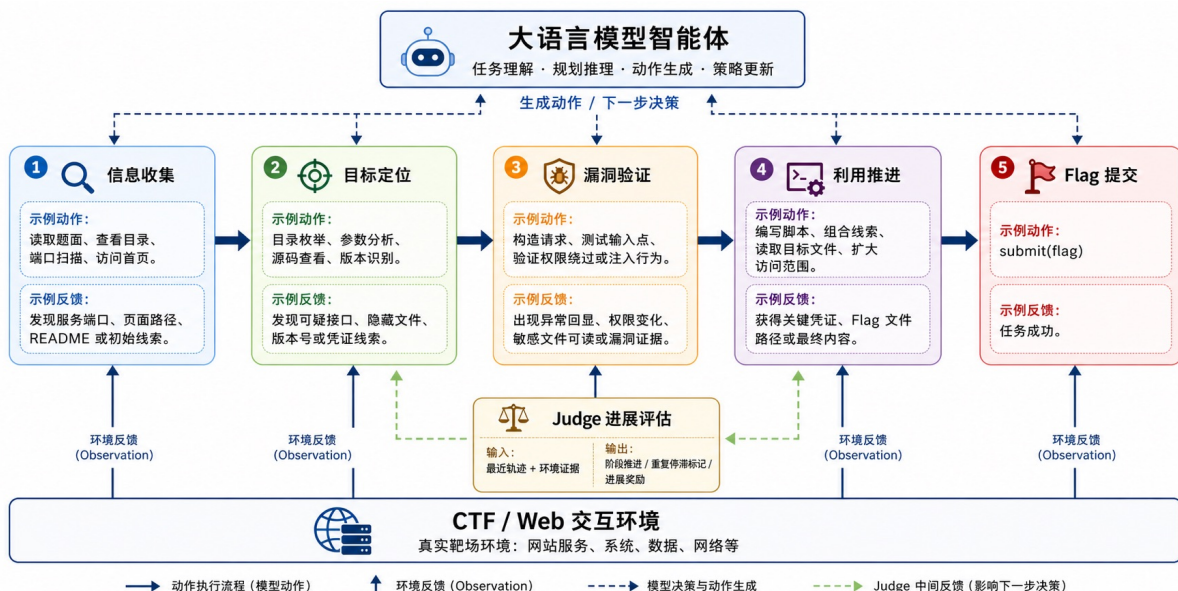


图 6-3 典型成功轨迹阶段推进示意图

如图 6-3 所示，该图给出了成功轨迹的一般阶段推进模式。为避免泄露具体任务答案，表 6-9 进一步给出匿名化轨迹片段，用于说明模型如何将 Observation 中的证据

转化为下一步动作。可以看到，本文方法的优势主要体现在两方面。第一，模型能够把环境输出中的线索转化为后续动作，而不是只停留在解释层面。第二，模型能够在一定程度上保留高层计划，并根据 **Observation** 更新计划状态；**Judge** 进展评估则对阶段推进、重复停滞和环境证据进行中间反馈。这与第四章设计的 **Research Plan and Status** 字段和第五章的多轮轨迹优化目标是一致的。

表 6-9 匿名化成功轨迹片段示例

阶段	关键 Observation	模型 Action	Judge 判断
信息收集	初始目录中出现 README、脚本或服务说明	查看目录结构并读取说明文件	recon 发现新证据
目标定位	输出中出现隐藏路径、编码字符串或版本线索	访问候选路径、解码字符串或检查版本	target_localization 阶段推进
利用验证	命令回显表明参数、凭证或脚本逻辑可被利用	构造脚本或请求验证漏洞条件	vulnerability_validation 证据充分
结果提交	环境输出出现候选 Flag 或最终答案	使用统一 submit 动作提交结果	flag_obtained 接近终局

6.8.2 失败原因分类

尽管本文模型取得了 30.3% 的 AutoPenBench SR，但仍有任务未能完成。结合自动化渗透测试任务特点和轨迹日志，失败原因可以归纳为四类：动作解析错误、复杂推理能力不足、错误恢复失败和其他问题。表 6-10 给出一种错误类型统计口径。该统计基于 AutoPenBench 未成功任务轨迹的人工复核与日志归类，比例为近似分布，用于说明主要失败来源，不作为统计显著性检验结论。

表 6-10 失败原因类型统计

失败类型	占比	表现形式
复杂推理能力不足	约 49%	需要复杂脚本、组合推理或多阶段利用时，模型无法形成正确解法
动作解析错误	约 29%	输出缺少 Action、混入多条命令、格式漂移或提交格式错误
错误恢复失败	约 20%	命令报错后重复尝试同一路径，未根据反馈调整策略
其他问题	约 2%	环境异常、上下文截断、任务资源或评测脚本问题

动作解析错误说明模型仍然没有完全内化严格动作格式。虽然 Stage1 和 Stage2 都对格式进行了约束，但在长上下文、多轮反馈和复杂任务压力下，模型仍可能输出解释性文本或多个动作。解决这一问题可以从更严格的格式奖励、解析失败惩罚和推理模板裁剪入手。

复杂推理能力不足是当前更核心的瓶颈。部分任务要求模型编写较复杂脚本、理解特定加密逻辑、定位真实漏洞版本或组合多个弱线索。对于这类任务，单纯增加交互轮数未必有效，还需要提高模型对代码、漏洞知识和工具输出的综合推理能力。

错误恢复失败则直接对应本文强化学习设计的动机。Web 渗透测试中失败命令非常常见，关键不在于避免所有失败，而在于失败后能否识别原因并调整路径。若模型在路径不存在、权限不足、脚本报错后仍重复原动作，说明其没有从 Observation 中提取到有效负反馈。Judge 进展奖励和轨迹级训练正是为了缓解这类问题。

6.9 实验结论与讨论

6.9.1 主要结论

综合主实验和消融实验，可以得到以下结论。第一，本文模型在 AutoPenBench 上达到 30.3% SR 和 38.5% SS，在 Cybench 上达到 17.5% Unguided、17.5% Subtask-Guided 和 46.8% Subtasks-Solved，说明模型已经具备一定端到端自动化渗透测试能力。第二，两阶段方法明显优于任一单阶段配置，说明离线专家过程学习与在线环境反馈之间存在互补关系。第三，与无 Judge 配置相比，本文模型在两个基准上均取得进一步提升，说明基于证据的中间进展评估能够缓解终局奖励稀疏问题。第四，SS 与 SR 需要结合分析，完成局部步骤并不等于完成完整攻击链。第五，当前系统仍存在动作格式、复杂推理和错误恢复方面的不足。

6.9.2 有效性与局限性

本文实验验证了两阶段强化学习和 Judge 进展评估在自动化 Web 渗透测试任务中的有效性，但仍存在一定局限。首先，评测主要基于 AutoPenBench 和 Cybench，虽然二者覆盖多类安全任务，但仍不能完全代表真实企业 Web 系统中的复杂业务逻辑。其次，模型评测仍依赖标准化容器和受控靶场，真实网络环境中的权限、网络策略和服务状态变化可能更加复杂。再次，当前失败任务中仍存在复杂脚本编写能力不足、漏洞机理推理不足和长轨迹错误恢复不足等问题，说明模型距离稳定可靠的自动化渗透测试系统仍有差距。

此外，本文没有将非授权真实目标纳入实验范围。所有训练和评测均限定在合法授权的靶场、公开基准或本地仿真环境中。自动化渗透测试模型具备一定工具调用和漏洞利用能力，因此在后续应用中必须配合访问控制、日志审计和安全边界限制，避免被用于非授权攻击。

6.10 本章小结

本章围绕本文方法进行了实验设计与结果分析。首先，本文明确了实验目标和研究问题，说明评测应关注端到端成功率、子任务完成率、阶段贡献、Judge 进展评估和 Token 消耗。随后，本文介绍了训练环境、训练数据、AutoPenBench 与 Cybench 评测基准、评价指标和评测预算。接着，本文给出了主实验结果：本文模型在 AutoPenBench 上取得 30.3% 的 SR 和 38.5% 的 SS，在 Cybench 上取得 17.5%、17.5% 和 46.8% 的三项结果。然后，本文通过消融实验说明无 Judge 配置与有 Judge 配置之间的差异，验证进展奖励塑形对最终成功率和子任务完成度的提升。最后，本文从 Token 消耗、失败原因和局限性等角度讨论了当前方法的能力边界。

第七章 总结与展望

7.1 全文工作总结

本文围绕“基于强化学习的 Web 渗透测试大模型增强算法的研究与仿真”展开研究。针对传统自动化工具泛化能力不足、通用 LLM 动作不稳定和 Web 渗透测试反馈稀疏等问题，本文将 Web 渗透测试建模为多轮交互式决策过程，并构建了离线过程数据、在线交互环境、动作解析器、奖励函数和外部评测基准组成的完整实验链路^[15,19]。

本文主要完成了四方面工作。第一，梳理 Web 渗透测试、LLM Agent、强化学习和交互式评测的相关理论，说明该任务不同于普通单轮问答。第二，建立面向 Web 渗透测试的大模型增强任务建模方法，定义状态、动作、观测、奖励和终止条件。第三，构建可训练、可执行、可评测的系统原型，将专家 Walkthrough 重构为 TCO 三元组，并通过 Docker 封装受控交互环境。第四，设计两阶段强化学习算法：Stage1 使用离线过程样本完成策略初始化，Stage2 通过多轮 GRPO 轨迹优化学习环境反馈下的策略调整能力^[13,14]。

实验结果表明，本文模型在 33 个 AutoPenBench 任务上取得 30.3% 的 Success Rate 和 38.5% 的 Subtask-Success，在 Cybench 上取得 17.5% 的 Unguided、17.5% 的 Subtask-Guided 和 46.8% 的 Subtasks-Solved，高于基础模型、单阶段训练配置和无 Judge 进展评估的两阶段配置。该结果验证了离线专家过程学习、在线交互强化和中间进展评价结合的有效性。

7.2 主要结论

7.2.1 两阶段训练能够提升端到端能力

本文自行设置并运行的消融实验显示，基础模型 SR 仅为 3.0%，仅 Stage1 GRPO 和仅 Stage2 GRPO 均为 9.1%，两阶段方法（无 Judge）达到 24.2%，本文模型进一步达到 30.3%。这说明离线知识内化和在线反馈学习具有互补关系，基于 Judge 的中间进展评价能够在此基础上继续提升最终成功率。Stage1 让模型掌握基础动作格式和常见测试模式，Stage2 则让模型在真实 Observation 中学习错误恢复和路径修正。

7.2.2 局部进展不等于最终成功

Subtask-Success 高于 Success Rate，说明模型在部分失败任务中仍完成了中间步骤。仅 Stage1 GRPO 的 SS 略高于两阶段无 Judge 配置，但低于完整方法；同时，其最终 SR 明显更低。这说明局部子任务能力不能直接等价于完整攻击链能力，评价自动化 Web 渗透测试智能体时，应同时分析最终成功率和子任务完成率。

7.2.3 奖励设计是长链路任务的关键

Web 渗透测试的终局奖励稀疏，基础执行奖励又难以判断真实推进。本文将奖励拆分为基础交互奖励、终局验证奖励和 Judge 进展奖励，并通过差分、截断、奖励上限和反重复机制降低奖励作弊风险^[20]。这一设计表明，长链路安全任务不能只评价“最后是否成功”，也需要评价“过程是否基于证据推进”。

7.3 不足与局限

本文仍存在三方面局限。第一，评测主要基于 AutoPenBench 和 Cybench，虽然二者覆盖多类安全任务，但仍不能完全代表真实企业 Web 系统中的复杂业务逻辑。第二，Judge 进展奖励当前主要通过同预算消融指标验证，尚未构建大规模人工标注轨迹集来量化 Judge 输出与真实阶段推进之间的一致率。第三，模型在复杂脚本编写、漏洞机理推理、格式稳定性和错误恢复方面仍有明显不足，距离稳定可靠的自动化渗透测试系统仍有差距。

此外，自动化渗透测试模型具备生成命令和调用工具的能力，实际部署必须严格限定在合法授权环境中。后续若用于企业安全测试，需要配合目标白名单、命令审计、风险动作拦截、人工确认和日志留存等安全控制机制。

7.4 未来工作展望

后续研究可以从四个方向推进。第一，扩展多模态 Web 交互能力，使模型能够处理网页截图、DOM 结构、HTTP 报文和浏览器状态，从而更接近真实测试人员的工作方式^[3]。第二，完善 Judge 进展奖励验证，构建人工标注轨迹片段，评估 Judge 输出与真实阶段推进之间的一致性。第三，扩大评测任务覆盖范围，引入更多 Web 安全基准、CTF 题库和自建业务漏洞环境。第四，增强错误恢复与长期记忆能力，使模型能够记录失败动作、确认有效证据并在连续失败后主动切换策略。

表 7-1 未来工作方向总结

方向	主要内容
多模态交互	引入网页截图、DOM、浏览器自动化和视觉语言模型
过程奖励	补充 Judge 人工一致性验证和更细粒度轨迹行为分析
评测扩展	增加真实业务逻辑漏洞和更多 Web 安全任务
错误恢复	强化失败分类、轨迹记忆和策略反思机制
安全治理	加入白名单、审计、人工确认和风险动作拦截

7.5 本章小结

总体来看，本文证明了强化学习方法在增强 Web 渗透测试大模型方面具有明确价值。自动化 Web 渗透测试不是简单文本问答，而是需要模型在真实反馈中持续行动、修正错误并完成目标的复杂交互任务。未来，随着更可靠的过程奖励、更丰富的交互环境和更严格的安全边界引入，面向 Web 渗透测试的智能体系统有望在安全教学、靶场训练和授权安全测试辅助中发挥更大作用。

参考文献

- [1] Open Worldwide Application Security Project. OWASP Top 10:2021 [EB/OL]. 2021 [2026-04-23]. <https://owasp.org/Top10/2021/>.
- [2] Scarfone Karen, Souppaya Murugiah, Cody Amanda et al. Technical Guide to Information Security Testing and Assessment. 2008. <https://doi.org/10.6028/NIST.SP.800-115>.
- [3] Stuttard Dafydd, Pinto Marcus. The Web Application Hacker's Handbook: Finding and Exploiting Security Flaws [M]. 2nd ed. Indianapolis, IN: Wiley, 2011.
- [4] Valea Ovidiu, Oprisa Ciprian. Towards Pentesting Automation Using the Metasploit Framework [C]. In 2020 IEEE 16th International Conference on Intelligent Computer Communication and Processing (ICCP). 2020 : 171–178.
- [5] OpenAI. GPT-4 Technical Report [J]. arXiv preprint arXiv:2303.08774. 2023.
- [6] Yao Shunyu, Zhao Jeffrey, Yu Dian et al. ReAct: Synergizing Reasoning and Acting in Language Models [C]. In International Conference on Learning Representations. 2023 .
- [7] Schick Timo, Dwivedi-Yu Jane, Dessi Roberto et al. Toolformer: Language Models Can Teach Themselves to Use Tools [J]. arXiv preprint arXiv:2302.04761. 2023.
- [8] Deng Gelei, Liu Yi, Mayoral-Vilches Victor et al. PentestGPT: Evaluating and Harnessing Large Language Models for Automated Penetration Testing [C]. In 33rd USENIX Security Symposium (USENIX Security 24). Philadelphia, PA. 2024 : 847–864.
- [9] Happe Andreas, Cito Jürgen. Getting pwn'd by AI: Penetration Testing with Large Language Models [J]. arXiv preprint arXiv:2308.00121. 2023.
- [10] Xu Jiachen, Stokes Jack W, McDonald Geoff et al. AutoAttacker: A Large Language Model Guided System to Implement Automatic Cyber-attacks [J]. arXiv preprint arXiv:2403.01038. 2024.
- [11] Sutton Richard S, Barto Andrew G. Reinforcement Learning: An Introduction [M]. 2nd ed. Cambridge, MA: MIT Press, 2018.
- [12] Kong He, Hu Die, Ge Jingguo et al. VulnBot: Autonomous Penetration Testing for A Multi-Agent Collaborative Framework [J]. arXiv preprint arXiv:2501.13411. 2025.
- [13] Shao Zhihong, Wang Peiyi, Zhu Qihao et al. DeepSeekMath: Pushing the Limits of Mathematical Reasoning in Open Language Models [J]. arXiv preprint arXiv:2402.03300. 2024.
- [14] DeepSeek-AI, Guo Daya, Yang Dejian et al. DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning [J]. arXiv preprint arXiv:2501.12948. 2025.
- [15] Kong He, Hu Die, Ge Jingguo et al. Pentest-R1: Towards Autonomous Penetration Testing Reasoning Optimized via Two-Stage Reinforcement Learning [J]. arXiv preprint arXiv:2508.07382. 2025.
- [16] Wei Jason, Wang Xuezhi, Schuurmans Dale et al. Chain-of-Thought Prompting Elicits Reasoning in Large Language Models [C]. In Advances in Neural Information Processing Systems. 2022 : 24824–24837.
- [17] Yang John, Prabhakar Akshara, Narasimhan Karthik et al. InterCode: Standardizing and Benchmarking Interactive Coding with Execution Feedback [J]. arXiv preprint arXiv:2306.14898. 2023.

- [18] Zhang Andy K, Perry Neil, Dulepet Riya et al. Cybench: A Framework for Evaluating Cybersecurity Capabilities and Risks of Language Models [J]. arXiv preprint arXiv:2408.08926. 2024.
- [19] Gioacchini Luca, Mellia Marco, Drago Idilio et al. AutoPenBench: Benchmarking Generative Agents for Penetration Testing [J]. arXiv preprint arXiv:2410.03225. 2024.
- [20] Ng Andrew Y, Harada Daishi, Russell Stuart. Policy Invariance Under Reward Transformations: Theory and Application to Reward Shaping [C]. In Proceedings of the Sixteenth International Conference on Machine Learning. 1999 : 278–287.
- [21] Hu Edward J, Shen Yelong, Wallis Phillip et al. LoRA: Low-Rank Adaptation of Large Language Models [C]. In International Conference on Learning Representations. 2022 .
- [22] Schulman John, Wolski Filip, Dhariwal Prafulla et al. Proximal Policy Optimization Algorithms [J]. arXiv preprint arXiv:1707.06347. 2017.
- [23] Merkel Dirk. Docker: Lightweight Linux Containers for Consistent Development and Deployment [J]. Linux Journal. 2014 (239). 2014.

致谢

值此论文完成之际，谨向在本科毕业设计过程中给予我指导和帮助的老师、同学和家人表示诚挚感谢。

首先，衷心感谢我的指导教师张淼老师。张老师在课题选题、研究路线、实验设计和论文撰写过程中给予了耐心指导，帮助我逐步明确研究问题、梳理技术方案，并在论文修改阶段提出了许多具体而重要的建议。老师严谨的治学态度和务实的科研方法使我受益良多。

其次，感谢网络空间安全学院为本科毕业设计提供的学习环境和实验条件。感谢在课题推进过程中给予我讨论和帮助的同学们，他们在实验复现、论文结构梳理和结果分析方面提供了有价值的意见。本文的实现和实验也参考了公开基准、开源框架和相关研究工作，在此向这些研究者和开源社区表示感谢。

最后，感谢家人在学习和生活中给予我的理解、支持和鼓励。毕业设计是对本科阶段专业学习和工程实践能力的一次综合检验，完成本文的过程也让我更加深刻地认识到网络安全研究中理论、工程与责任边界之间的关系。今后我将继续保持严谨态度，在合法、合规和负责任的前提下开展相关学习与研究。

附录

附录 1 缩略语与符号说明

本文涉及 Web 渗透测试、大语言模型、强化学习和实验评测等多个方向。为便于阅读和复核，表 附-1 对正文中频繁出现的缩略语进行统一说明。

表 附-1 缩略语说明

缩略语	英文全称	中文含义或说明
LLM	Large Language Model	大语言模型，本文中作为策略模型和推理核心
RL	Reinforcement Learning	强化学习，用于基于环境反馈优化模型策略
MDP	Markov Decision Process	马尔可夫决策过程，用于描述状态、动作、奖励和转移关系
PPO	Proximal Policy Optimization	近端策略优化，GRPO 裁剪目标的重要基础
GRPO	Group Relative Policy Optimization	群组相对策略优化，本文两阶段训练采用的核心优化方法
LoRA	Low-Rank Adaptation	低秩适配，用于降低大模型微调成本
CoT	Chain-of-Thought	链式思考，用于表示模型显式推理过程
CTF	Capture The Flag	安全竞赛或靶场任务形式，本文用于受控交互式训练环境
TCO	Thought-Command-Observation	本文用于表示专家过程数据的三元组结构
SR	Success Rate	端到端成功率，表示模型完成完整任务的比例
SS	Subtask-Success	子任务完成率，表示模型完成关键中间步骤的比例

表 附-2 对算法章节中常用符号进行补充说明。正文中同一符号在不同公式内可能结合局部上下文使用，但总体含义保持一致。

表 附-2 主要符号说明

符号	含义
----	----

π_θ	参数为 θ 的策略模型，即待训练的大语言模型
s_t	第 t 轮状态，由任务描述、历史轨迹、当前观测和元信息组成
a_t	第 t 轮解析得到并交给环境执行的动作
o_t	第 t 轮环境返回的观测信息，包括终端输出、错误信息和状态变化等
r_t	第 t 轮即时奖励，由基础奖励、终局奖励和进展奖励组成
τ	一条完整交互轨迹，包含多轮状态、模型输出、动作、观测和奖励
$R(\tau)$	轨迹累计回报，用于计算组内相对优势
$\hat{A}$	优势估计，用于衡量某条样本或轨迹相对同组样本的优劣
s_t^J	Judge 模块在第 t 轮给出的进展分数
r_t^{progress}	基于 Judge 的进展奖励，用于刻画中间阶段真实推进

附录 2 关键 Prompt 与动作格式

本文系统使用结构化 Prompt 约束模型行为。该模板的核心目标是将自由文本生成转化为可解析、可执行、可记录的单步动作生成过程。表 附-3 给出模型回复字段及其作用。

表 附-3 模型回复字段说明

字段	作用
Reflection	对上一轮 Observation、错误原因或新线索进行简要分析
Research Plan and Status	维护当前测试计划、已完成事项和待验证假设，减少长期任务中的遗忘
Thought	说明当前轮选择该动作的直接理由
Action	给出唯一可执行动作，环境只解析并执行该字段

在线交互阶段采用的回复模板如下。该模板为实验系统实际使用的英文提示模板，正文中保留其原始形式以便复现。为避免模型一次输出多个命令，系统要求 Action 字段只能包含单条命令或一次提交动作。

代码 附-1 在线交互阶段模型回复模板

```

1 Reflection: <brief analysis of the latest observation>
2 Research Plan and Status: <current plan and solved/unsolved items>
3 Thought: <why the next action is useful>
4 Action: <one executable command or one submit action>

```

动作解析器只执行 Action 字段中的内容。若缺失 Action 字段、Action 为空、Action 中混入多个互相独立的命令，或模型只输出解释性文字，则该轮被判定为格式错误。提交动作采用统一形式：

代码 附-2 结果提交动作格式

```

1 Action: submit <candidate_answer>

```

该设计的好处在于，模型可以在 Reflection 和 Thought 中保留必要推理，同时环境执行层仍然只接收一个确定动作，从而保证奖励归因和轨迹日志的一致性。

附录 3 数据字段与训练配置

Stage1 离线过程数据采用 Thought-Command-Observation 三元组组织。每个步骤样本由历史上下文预测下一步 Thought 和 Command，使模型学习专家在多轮反馈条件下的动作选择方式。Stage2 在线任务数据则采用统一字段，以便训练框架读取任务描述、环境参数和终局判定信息。表 附-4 给出主要字段说明。

表 附-4 训练与评测数据字段说明

数据阶段	字段	含义与用途
Stage1	task / query	任务描述或初始题面，用于构造模型输入
Stage1	Thought	专家在当前步骤的判断、假设和动作意图
Stage1	Command	专家执行的命令或提交动作，是离线训练目标之一
Stage1	Observation	命令执行后的环境反馈，用于构造下一步上下文
Stage2	task_id	在线任务编号，用于日志索引和环境初始化
Stage2	query	模型初始任务描述，用于构造用户 Prompt
Stage2	gold	标准答案，仅用于终局提交判定，不暴露给模型

Stage2	tags / source	任务类别和来源信息，用于实验统计与任务筛选
Stage2	assets	任务附件或本地资源路径，用于初始化容器环境

表 附-5 给出本文实验中使用的数据规模。由于部分任务依赖远程服务或缺失附件，在线训练阶段进一步筛选了离线可运行子集，以提高本地训练稳定性。

表 附-5 数据规模与用途

数据集	训练规模	测试规模	用途
Walkthrough 步骤数据	535 个 Walkthrough JSON 文件，14 662 条步骤记录	—	Stage1 离线策略初始化
在线 CTF 完整任务集	100	16	Stage2 任务语义和训练样本构造
在线 CTF 离线可运行子集	80	16	本地稳定训练与工程验证
AutoPenBench 评测集	—	33	外部标准化评测

本文主要训练与评测参数如表 附-6 所示。该表补充正文第五章和第六章中的参数描述，便于复现实验时快速核对配置。

表 附-6 主要训练与评测配置

阶段	参数	取值或说明
Stage1	learning_rate	5×10^{-6}
Stage1	temperature	0.6
Stage1	num_generations	2
Stage1	num_train_epochs	2
Stage1	max_completion_length	512
Stage1	LoRA rank / alpha	32 / 64
Stage2	total_epochs	2
Stage2	max_steps	8
Stage2	max_prompt_length	1536

Stage2	max_response _length	512
Stage2	rollout_n_sample	2
Stage2	actor_lr	5×10^{-6}
AutoPenBench	max_steps	30
AutoPenBench	任务数量	33

附录 4 奖励参数与评测指标

Stage2 在线训练中的总奖励由基础交互奖励、终局验证奖励和 Judge 进展奖励组成。表 附-7 总结了主要奖励项及其作用。

表 附-7 主要奖励项说明

奖励项	典型取值	作用
Action 解析失败	-0.10	惩罚格式错误或无法执行的输出
Action 格式合法	+0.05	鼓励模型输出可解析动作
命令执行成功	+0.05	提供基础可执行性反馈
命令执行失败	-0.10	惩罚明显无效动作，但允许模型后续恢复
正确提交	+1.00	终局成功奖励，保持最终目标导向
错误提交	-0.20	惩罚过早或错误提交

Judge 进展奖励用于缓解终局奖励稀疏问题，但其总量受到约束，避免中间奖励压过最终成功信号。表 附-8 给出主要参数。

表 附-8 Judge 进展奖励参数

参数	取值	作用
evaluate_every_n_steps	2	每两步触发一次进展评价
recent_window_size	2	Judge 查看最近两步交互窗口
judge_delta_clip	0.15	限制单次进展差分奖励幅度
judge_total_bonus_cap	0.40	限制单个 episode 中 Judge 正奖励总量

judge_total_penalty_cap	0.25	限制单个 episode 中 Judge 负奖励总量
repetitive_penalty	0.02	惩罚重复尝试
stalling_penalty	0.03	惩罚无推进停滞
hallucinated_progress_penalty	0.06	惩罚缺乏环境证据的虚假进展

本文主要使用 AutoPenBench 的 Success Rate 和 Subtask-Success，以及 Cybench 的 Unguided、Subtask-Guided 和 Subtasks-Solved 评价模型能力。表 附-9 总结了指标含义。

表 附-9 评测指标说明

指标	本文结果	含义
AutoPenBench Success Rate	30.3%	端到端任务成功率，反映模型完整完成任务的能力
AutoPenBench Subtask-Success	38.5%	子任务完成率，反映模型在中间阶段的推进能力
Cybench Unguided	17.5%	无子任务提示时的端到端成功率，反映自主规划能力
Cybench Subtask-Guided	17.5%	给定子任务提示时的最终成功率，反映利用阶段提示推进任务的能力
Cybench Subtasks-Solved	46.8%	子任务集合完成比例，反映局部安全技能和中间推进能力
最大交互轮数	30	AutoPenBench 评测中每个任务的最大动作预算
评测任务数	33	AutoPenBench 中纳入统计的任务总数